Chapter 1

Introduction

XSPEC is a command-driven, interactive, X-ray spectral-fitting program, designed to be completely detector-independent so that it can be used for any spectrometer. XSPEC has been used to analyze data from HEAO-1 A2, Einstein Observatory, EXOSAT, Ginga, ROSAT, BBXRT, ASCA, CGRO, IUE, RXTE, Chandra, XMM-Newton, Integral/SPI, Fermi, Swift, Suzaku, NuSTAR, AstroSat, Hitomi, Insight-HXMT, NICER, IXPE, Spectr-Roentgen-Gamma, and XRISM. There are now more than 15,000 refereed papers listed on ADS which mention XSPEC.

This manual describes XSPEC v13, which is available on Linux and MacOSX. The Python interface PyXspec is described in a separate manual at https://heasarc.gsfc.nasa.gov/docs/software/xspec/python/html/index.html. New users are advised to read Chapter 2, which introduces spectral fitting and the XSPEC approach, Chapter 4, which gives an overview of the program commands, and Chapter 5, which contains walkthroughs of XSPEC sessions. Chapter 3 describes the Bayesian methods available in XSPEC — priors, Markov Chain Monte Carlo, Hamiltonian Monte Carlo, nested sampling, and model comparison. Users should then experiment with XSPEC and, if necessary, look up individual commands in Chapter 6, or descriptions of the spectral models in use, in Chapter 7.

The user interface for XSPEC, which employs a tcl scripting shell and the XSPEC parser, is described in Appendix A. The statistical methods used within XSPEC are described in Appendix B. Users interested in adding their own models can read how to do so in Appendix C. Appendix D describes the PLT plotting package which XSPEC currently uses for graphical output. Some of the tools (FTOOLS, programs, scripts) that can operate on XSPEC files are listed in Appendix E. The XSPEC model library can be linked into other programs

following the instructions in Appendix F. Appendix G describes how to use your own proposal distribution for Markov Chain Monte Carlo. Appendix H describes the numerical algorithms underpinning fitting and sampling — the analytic-gradient pipeline and response compression. Appendix I lists the release notes for the current and older versions of XSPEC. Appendix J defines some commonly used XSPEC terms. Finally, Appendix K includes some notes on the changes between XSPEC v11 and v12.

1.1 New in version 13.0.1

Version 13.0 is a major release whose principal theme is Bayesian inference and global optimization. A new chapter on Bayesian methods (Chapter 3) and a new Algorithms appendix (Appendix H) describe the analytic-gradient and response-compression machinery that underpins many of the features below.

Bayesian inference and optimization

- Closed-form analytic derivatives (Jacobians and their reverse-mode vector-Jacobian products) are now available for the great majority of the model library. These provide a fast Levenberg-Marquardt path, in which the gradient is computed analytically in a single pass rather than by finite differences, and supply the posterior gradient used by the new Hamiltonian Monte Carlo sampler. The pipeline is described in Appendix H.1.
- New **hmc** command: a No-U-Turn-Sampler variant of Hamiltonian Monte Carlo over the thawed parameters, using the analytic posterior gradient. It supports warmup adaptation of the leap-frog step size and diagonal mass matrix, multiple independent chains (parallelized with **parallel hmc**), and checkpoint/resume. Output is written in a format compatible with **chain** `load`.
- New **nest** command: a multi-ellipsoidal nested sampler that produces weighted posterior samples together with an estimate of the marginal likelihood (model evidence, $\log Z$). It supports checkpoint/resume and parallel sampling (**parallel nest**).
- New **compare** command: automated model selection. Alternative models are evaluated against the current data and ranked by the Akaike and Bayesian information criteria, by Bayesian evidence (**compare** `nest`), or by fitting each alternative with the global optimizer (**compare** `global`).
- New **fit global** option: a derivative-free global optimization by Differential Evolution followed by a polish with the current fit method. It is never-regress (the original fit is restored if no improvement is found),

reproducible under **xset seed**, parallelizable with **parallel global**, and reports any competitive minima within Δstat < 9 of the best fit. The **improve** command now uses the same Differential Evolution machinery as a local warm-restart and works with all fit methods, not just the Minuit methods.

- The Minuit fit methods (`migrad`, `simplex`) were overhauled: Minuit now works directly in the parameter's stored value (so a fit started between a parameter's soft and hard limits reaches the same minimum as one started inside them), hard limits are passed to Minuit only when they span a range of 100 or less, and a fit that stops short of the minimum now says so instead of reporting convergence — with different warnings for running out of function evaluations (fit again to continue) and for Minuit finding no direction of improvement. See the **method** command.
- New **bayes** `smooth` subcommand: a multi-parameter smoothness prior that couples a chosen parameter across datagroups, penalizing shell-to-shell roughness. It is the in-engine regularizer for **projct** de-projection, with automatic selection of the regularization strength by the L-curve or discrepancy principle. Several new single-parameter prior families were also added (cauchy, beta, studentst, halfnormal, powerlaw, isotropic, log-normal, gamma, and shiftedlognormal).
- Added an MCMC diagnostics suite, including the Rubin-Gelman $\hat{R}$ convergence statistic, and gave **chain**, **hmc** and **nest** a common sample format.
- PyXspec gained a unified Bayesian-inference module (runHMC, runChain, shared result objects, and arviz interoperability), together with `Fit.compare` and `Fit.globalFit`.

New fit statistics

- `chistokes`: a chi-squared variant that uses the per-bin Stokes Q/U cross-covariance when fitting polarization spectra, rather than treating Q and U as independent. It has been generalized to an arbitrary n-member cross-spectrum covariance group (with the `chicov` alias), so any set of bin-by-bin correlated spectra — overlapping extraction regions, grating orders, or the full Stokes I/Q/U triple — can be fit jointly with a multivariate-Gaussian likelihood, per-bin Cholesky solve and graceful shrinkage of non-positive-definite covariances. The covariances are carried in the spectrum files (`COVFRMT='XCOV-SET-1'`; legacy Q–U pair files are auto-promoted), produced by the **ftextractor** tool (`stokescov=iqu`) and summed under grouping by ftgrouppha. See Appendix B.

New models

- Four X-ray spectropolarimetric reflection models based on precomputed `STOKES` tables, designed for joint fitting of the Stokes $I/Q/U$ spectra: **stdisc** (reflection of a power-law source of arbitrary incident polarization from the distant, nearly neutral equatorial regions of an accretion disc), and **sttorus**, **stcone** and **stbowl** (reflection from an optically thick, axially symmetric torus, double cone, or double bowl – representing e.g. an opaque AGN torus, a broad-line region, or a super-Eddington accretion funnel – with partially ionized, neutral, or Chandrasekhar electron-scattering surfaces).

Modifications to models

- The broadening of the APEC/SPEX pseudo-continuum (set by the BROADPSEUDO xset option) now uses an FFT and accepts `fast` and `slow` options. The `fast` option (equivalent to `on`) uses an abundance-weighted atomic mass so that a single FFT broadens all elements at once, about five times faster than the per-element `slow` calculation.

- The **gsmooth** and **lsmooth** convolution models use an FFT for the index 0 and index 1 cases, giving order-of-magnitude speed-ups, and cache the kernel FFT when it has not changed.

- Added a check that warns (yellow alert) when the fiducial energy at which a line width is specified falls outside the input energy range.

- Table models (**atable**, **mtable**, **etable**) may now carry a neural network (NN) emulation of the tabulated spectra, indicated by an NNPTFILE keyword in the table file pointing to a PyTorch (`.pt`) weights file. When such a table is loaded XSPEC automatically evaluates the model with the neural network instead of interpolating on the parameter grid, with no change to how the model is set up. This requires XSPEC to have been built with libtorch (the `--with-libtorch` configure option).

- The **crossarf** mixing model can now be stacked, one copy per source, so that a single sky region may host several emission components with different spatial distributions – for example an extended source and a point source, which spill into neighbouring regions by different fractions and so require separate arfs. Each source's cross arfs are supplied by source-tagged XFLT keywords of the form "CrossArfSrcSFromNToM" (source S, contribution to region M from region N); the original "CrossArfFromNToM" form is unchanged and applies to source 1.

- The **recorn** mixing model is once again applied separately within each data group, so the correction norms of several spectra can be fitted at the same time. Its first parameter names a single spectrum, so unlike the other mixing models – whose one transformation spans every data group

– **recorn** needs one copy per corrected spectrum. Each data group's copy now has its own spectrum number and cornorm, all of which appear in **show** `par` and can be set and fitted; in the intervening releases only the lowest data group's copy did anything.

- The lower limit on **clumin**'s <`Redshift`> parameter has been increased from −0.999 to 0.

Other

- Spectral model data files are no longer shipped with HEAsoft. The new `ftgetmodeldata` tool downloads, on demand, the data files required by the models in use, over the network (via cURL), with version lookup (including a `latest` option), a manifest file (`modelDataFiles.csv`), and clobber and dry-run options. See https://heasarc.gsfc.nasa.gov/docs/software/xspec/modeldata.html.
- New **group** command for runtime (re)grouping of spectra without rereading the files. Its most important use is **group** `back`, which lays a coarser "super-bin" grouping *on top of* the current source binning — each super-bin spanning a whole number of source bins — to reduce a bias in the C statistic when a background is present (the W statistic, see Appendix B). That statistic profiles out one background-rate nuisance parameter per source bin; when the background has few counts per bin these estimates are noisy and bias the fitted source parameters — an incidental-parameters problem that can reach tens of percent for faint sources on a low-count background. Sharing a single background rate across the source bins of each super-bin shrinks the number of nuisance parameters and can bring the bias down to a few percent. The pooled-background path is selected automatically whenever a background super-bin grouping has been set up at runtime with **group** `back`; a `GROUPING` column carried by the background file (e.g. from `ftgrouppha`) is not used for this, since on input XSPEC applies the source grouping to the background. The command also regroups the source spectrum itself (**group** `data` with `optbin`, `minsn`, `mincounts`, `const`, or `file`); all grouping is persisted in **save** files, recoverable with **tclout** `group`, and available in PyXspec.
- New **sim** command for bulk posterior-predictive simulation: it draws many parameter sets (from the loaded chains, or from the fit covariance scaled by **sim** `fsigma`), realizes each as a simulated dataset in the loaded spectra, optionally refits it (a parametric bootstrap), and accumulates the results — the in-program replacement for the scripted **tclout** `simpars` + **newpar** + **fakeit** loop (the **multifake** script). Because the realization happens in place no files are written and the original state is restored on completion. An optional command file (**sim** `@cmdfile`) is run on every realization and may return a row of numbers or append to a per-realization

output file; the products are recovered with **tclout** `sim`. The whole run is reproducible under **xset seed** and parallelizable with **parallel sim**.

- The **plot** command allows finer control over plot stacks by using the pipe (`|`) character to delimit them, e.g. `plot da res | mo | ld rat` draws three stacks. At present up to four stacks may be drawn, each holding up to four panes.
- New **plot** options `dspec`, `despec` and `deespec`: the model-independent counterparts of `ufspec`, `eufspec` and `eeufspec`. The flux density is obtained by a regularized truncated-SVD inversion of the detector response rather than by scaling the data by the model-dependent unfolded/folded ratio, so the plotted data points do not move when the model is changed.
- Operational switches of the form **xset** `CONTROL_STRING yes` control the numerical paths used in fitting and model evaluation and enable cross-check validation modes. For example, `DISABLE_ANALYTIC_GRAD` reverts fits to finite-difference derivatives. They are listed in a new table in the **xset** entry, separate from the model control strings.
- **chain** `info` and **margin** accept a parameter number of 0 to report and plot the distribution of the statistic values stored in a chain.
- The **identify** command gained a `spex` option to search the SPEX line list.
- Python local models are now supported in the standalone (Tcl) version of XSPEC, not only in PyXspec. A Python model function may also opt into a faster vectorized interface with the `@xspec.vectorized` decorator, receiving zero-copy `numpy` arrays instead of Python tuples and lists — avoiding the per-element Python-object overhead (Appendix C).
- Rank-K SVD compression of response matrices is now supported: a compressed side-car file (produced by the `ftsvdcmprmf` tool) can be loaded with the **response** command and tuned with the `svdrank` and `svdeps` options for fast folding (Appendix H.2).
- The XSPEC libraries were reorganized so they can be linked into third-party C++ programs without Tcl or PGPLOT: core start-up was split into a new XSCore library, and the model and data classes were split into separate XSModel and XSData libraries. The PLT plotting package was modernized to call PLT and PGPLOT directly from C++.

Fixes

Many user-visible bugs present in earlier versions have been fixed. The more significant fixes are grouped by area below; a complete list of the user-facing fixes in this release is given in Appendix I.

Models

- The **btapec** model overwrote the helium abundance with the metal abundance, producing a wrong continuum whenever the abundance parameter was not 1.
- The **zvgabs** absorption-line width was applied in km/s instead of the equivalent keV value.
- The velocity-broadened Lorentzian and Voigt models (the **vlorentz** and **vvoigt** families) shipped with a width-parameter hard maximum of 20 km/s, below their own default of 100 km/s, so a fit could never raise the width above 20 km/s. The maxima now match the **vgauss** family.
- The **xmmpsf** and **suzpsf** PSF mixing models set the model flux to zero above 6 keV.
- Out-of-bounds memory writes in the **diskline** model (when a line's blue edge fell in the first energy bin) and the **pileup** model (when `max_ph` = 1) could corrupt memory or crash XSPEC.
- The **vvnpshock** and **vvsedov** models (and their **bvvnpshock**/**bvvsedov** variants) reused stale ionization fractions when only the electron temperature was changed, giving a wrong spectrum.
- The **nteaa** model could crash for some combinations of parameter values. This has been fixed and there are no changes for combinations of parameter values which previously worked.

Data, responses and setup

- Defining a model with more than one data group loaded read out of bounds, aborting XSPEC on builds compiled with array-bounds assertions.
- An **ignore** or **notice** command whose channel range lay entirely above the spectrum's channel count could crash.
- Deleting several spectra in a single **data** command could remove the wrong spectra.
- A table-model file path containing a slash or a hyphen was corrupted when the component was added with **addcomp**.
- Channels re-noticed after a **dummyrsp** command were incorrectly folded to zero.
- Variable-length QUALITY and GROUPING columns were incorrectly assumed to be scalar when their repeat count was 1.

Fitting and statistics

- A segmentation fault in the `cstat` statistic when all channels are ignored.
- The default Levenberg-Marquardt (`leven`) method could leave parameters stuck pegged at a hard limit.
- A parameter resting on one of its limits was given a numerical derivative half its true value, so the Levenberg-Marquardt step for it was twice as long as it should be. Additive norms met this routinely. On the paths that differentiate the statistic directly (**xset** `usechainrule no`, and the **recorn** cornorm) the same parameter could hang **fit** outright.
- **fit** could abort the whole session, or read out of bounds with a Minuit method, when the covariance matrix was ill-conditioned or not positive-definite; it now issues a recoverable error.
- The empirical distribution function test statistics could crash or return NaN: with every channel ignored, with a model integrating to zero over the noticed range, or with a spectrum that was folded but not fitted.
- The **recorn** cornorm parameter could not be fitted: because it rescales the data rather than the model, the chain-rule derivative was identically zero, so it was pegged as insensitive and **fit** stopped with "No variable parameters for fit".
- **untie** could free a mixing-model parameter outside the lowest data group, producing a parameter that could not be displayed, set or fitted but still consumed a degree of freedom.
- The continue fitting question in a **fit** did not correctly handle an end of input from a piped script or redirected input file leading to a possible infinite loop if **query** was set to on.
- An **error** calculation that found a new minimum restarted itself without checking that re-fitting had actually improved the statistic. On a fit that was not properly converged the same non-existent minimum was found repeatedly leading to an infinite loop.
- Independent processes started within the same second shared identical pseudo-random number streams because the generator was seeded from the clock at one-second resolution. Simulations run in parallel from a script – **fakeit**, **goodness**, and the parallel workers used by **chain** – could therefore repeat one another exactly. The seed is now drawn from system entropy, and the parallel +workers derive distinct streams from it. Reproducibility on demand is still guaranteed by using **xset** `seed`.
- Under some (unusual) circumstances it was possible for **error** and **rerror** to silently report the confidence interval of the wrong parameter.

Plotting

- **plot** `sensitivity` and `insensitivity` drew the first detector's curve for every response of a multiple-response spectrum.
- **plot** `delchi`, `ufspec`, `fitstat` and `icounts` could crash for a plot group whose spectrum has no response for an active model source.
- The polarization plots `polfrac` and `polangle` returned NaN or zero for multiple-source models and could contain spurious or infinite bins.

Commands and scripting

- **autosave** `0` crashed XSPEC with a divide-by-zero on the next command.
- Running a script by a path containing a directory component (for example `@../model`) failed to append the default `.xcm` suffix.
- **energies** with the `log` option and a low energy of 0 silently built a degenerate energy grid for every model.

Fixes for bugs introduced in 13.0.0

- An **ignore** or **notice** range in energy or wavelength which mapped to only one channel had no effect.
- The **nest** command crashed xspec with an invalid numerical argument (e.g. a negative number of live points).

1.2 How to find out more information

XSPEC is distributed and maintained under the aegis of the GSFC High Energy Astrophysics Science Archival Research Center (HEASARC). It can be downloaded as part of HEAsoft from
https://heasarc.gsfc.nasa.gov/docs/software/lheasoft/download.html

XSPEC is available either as binaries or source. We recommend downloading the source and compiling locally to avoid potential system library conflicts and allow installation of any patches we release. If you have any problems compiling or running XSPEC please e-mail us at xspechelp@athena.gsfc.nasa.gov.

Chapter 3

Bayesian Methods

3.1 Introduction

Most of the discussion in Chapter 2 treats spectral fitting as a maximum-likelihood problem: find the single parameter vector $\hat{\boldsymbol{\theta}}$ that minimizes the fit statistic ($-2\ln L$), and report confidence intervals from the curvature of the statistic around that minimum. This chapter covers the alternative Bayesian view, in which the goal is the full posterior distribution

$$p(\boldsymbol{\theta}|D) \;\propto\; p(D|\boldsymbol{\theta})\,p(\boldsymbol{\theta}), \tag{3.1}$$

where $p(D|\boldsymbol{\theta})$ is the likelihood given by the chosen fit statistic, $p(\boldsymbol{\theta})$ is the prior on the parameters, and D is the observed data. Once the posterior is in hand, every downstream quantity — best-fit point, credible interval, derived-quantity distribution, evidence for model comparison — is just an expectation or quantile against $p(\boldsymbol{\theta}|D)$. The work shifts from *minimizing* a function to *sampling* from a distribution.

XSPEC has built up a complete Bayesian toolchain over the last several releases. The user supplies priors with the **bayes** command and then chooses one of four samplers: Metropolis–Hastings, Goodman–Weare, Hamiltonian Monte Carlo, or nested sampling. The first two are accessed through the **chain** command; the third and fourth have their own **hmc** and **nest** top-level commands. The four are not interchangeable: each is well-suited to a different regime, and the choice has a much larger effect on elapsed time and convergence reliability than any single tuning knob within a given sampler. This chapter explains what each one does, when to reach for it, and which mistakes to avoid.

3.2 Priors

3.2.1 Why priors matter, and when they don't

When the data are abundant and informative, the likelihood is sharply peaked and the posterior is almost independent of any reasonable prior. In that limit a Bayesian fit and a maximum-likelihood fit agree to within the precision of the sampler. When the data are sparse or the model has many parameters per bin — low signal-to-noise sources, partially degenerate components, broad-band CCD spectra with strong absorption — the prior carries real weight and the posterior depends on what was assumed. Reporting the prior choice alongside the result is then mandatory; choosing it carelessly is just as problematic as ignoring background subtraction.

Two practical consequences follow. First, every parameter that the user is genuinely uncertain about must have a prior set explicitly. The default of a constant (flat) prior over the parameter's full allowed range is rarely appropriate when the allowed range spans many decades, because most of the prior mass then sits in the exponentially unlikely regions and pulls the inference towards them. Second, the prior interacts with the parameter's hard limits set through the **newpar** command. Setting an absurd hard upper limit "just to be safe" is harmless under maximum likelihood, but it can dominate a posterior that has heavy tails.

3.2.2 Available prior families

The full set of priors recognized by the **bayes** command is listed in the command reference; the table there gives the $\ln p(\theta)$ expression for each. The shapes most often useful for X-ray work fall into three classes:

- **Scale parameters** — normalizations, column densities, and other quantities that are bounded below by zero and whose order of magnitude is itself uncertain. Use `jeffreys` (uniform on $\log\theta$), or `lognormal` if you have a preferred order of magnitude with a width on $\ln\theta$. Both require strictly positive support; the parameter's hard lower limit must be greater than zero before either is selected.

- **Location parameters** — temperatures, line centroids, photon indices, and other quantities defined on a finite linear range. Use `gauss` (or `cauchy` if you want heavier tails to avoid undue shrinkage), or `cons` if the parameter is well-constrained by the data alone over its allowed range.

- **Bounded parameters** — covering fractions, abundances normalized to unity, and angles. Use `beta` on $[0, 1]$, `isotropic` for an angle drawn from a uniform direction on the sphere, or `powerlaw` for a truncated power-law distribution.

Two further specialized forms are available: `shiftedlognormal` for quantities bounded below by a non-zero value (useful for black-hole spins displaced from zero), and `halfnormal` for a mildly informative prior on a positive parameter without committing to a particular scale. `exp` and `gamma` are useful when prior knowledge takes the form of a rate parameter or shape-and-rate pair from a previous analysis.

3.2.3 Practical advice

For most XSPEC users beginning a Bayesian analysis, the following default choices work well:

- Set `jeffreys` on every normalization and on any column density whose value crosses multiple decades. Use **newpar** first to push the parameter's hard lower limit to a small positive value (e.g. 10^{-6}) if it is currently zero.
- Set `gauss` or `cauchy` on physical parameters for which a published value or theoretical expectation exists, using the published 1σ uncertainty as the prior width.
- Leave `cons` on parameters whose hard limits are already a fair representation of physical possibility (e.g. photon index restricted to $0 < \Gamma < 5$).
- Switch Bayesian inference on with **bayes** `on` once all priors are in place. The samplers will not consult the prior unless this is set.

Hyperparameters — the $h_1, h_2, \ldots$ that appear in the prior expressions — should be drawn from prior knowledge or pilot fits, not tuned to the same data the posterior will be conditioned on. Doubly using the data corrupts the credible-interval coverage.

3.2.4 Joint priors: the smoothness regularizer

All the priors above act on one parameter at a time. Some inference problems are instead under-determined in a way that only a prior *coupling several parameters* can fix. The canonical X-ray example is spatially-resolved spectroscopy with

the **projct** model: the de-projection that recovers the emissivity of each 3-D shell from the projected annular spectra is an ill-conditioned inverse, so the per-shell normalizations come out noisy and strongly anti-correlated, and finer radial binning makes it worse rather than better. No per-parameter prior helps, because the problem is not that any single shell is unconstrained — it is that neighboring shells trade emission back and forth.

The **bayes** `smooth` command supplies the standard remedy, a Tikhonov smoothness prior. It couples a chosen model parameter (the normalization, say, or the temperature) across the datagroups and penalizes shell-to-shell roughness,

$$\ln p(\boldsymbol{\theta}) = -\tfrac{1}{2}\,\lambda\,\|L\,\boldsymbol{q}\|^2, \qquad q_k = p_k \text{ or } \ln p_k, \tag{3.2}$$

where L is a spacing-aware first- or second-difference operator built from the shell radii (penalizing the gradient or the curvature of the profile respectively), and λ sets the strength. This is exactly a zero-mean Gaussian prior on the shell parameters with precision $\lambda L^{\mathsf{T}} L$ — a *correlated* prior, the first whose density does not factorize over the parameters. The radii are taken automatically from each datagroup's XFLT `major` keyword, so for a **projct** fit the whole prior is set with a single line:

```
bayes smooth 2 order 2 space log lambda 5
```

The strength can be fixed, or selected automatically at the corner of the misfit-versus-roughness L-curve (`lcurve`) or by the discrepancy principle (`discrep`); the latter raises the smoothing until the data misfit equals the number of degrees of freedom.

Because the prior enters through the same $\ln p(\boldsymbol{\theta})$ that the per-parameter priors use, it regularizes everything downstream with no further work: the Levenberg–Marquardt fit and its error bars, and all four samplers below. Under **nest** the prior of Eq. 3.2 needs one extra step, because certain profile shapes leave the penalty unchanged: adding the same constant to every shell does not alter the differences a first-difference L measures, and a constant or a linear-in-radius ramp does not alter the curvatures a second-difference L measures. For those components $\|L\,\boldsymbol{q}\| = 0$ (they span the null space of L, and make the precision matrix $\lambda L^{\mathsf{T}} L$ rank-deficient), so the prior expresses no preference about them at all. The prior is therefore sampled through a spectral transform: a Gaussian draw along each penalized eigen-direction, and a flat draw, within the parameter limits, along the unconstrained directions — the overall level and, for second differences, the overall slope. The posterior then shows the adjacent shells smoothed together while the overall amplitude of the profile stays free.

The smoothness prior is the first concrete case of a more general multi-parameter prior facility; the per-parameter priors of the previous section are simply its

one-parameter limit. Settings round-trip through **save**/`@file` and are listed by **show** `bayes` (PyXspec `Fit.bayesPriors`).

3.3 Samplers

XSPEC implements four samplers, partitioned naturally by what they need from the model and what they return. The first three target the posterior directly; the fourth additionally returns the model evidence $Z = \int p(D|\boldsymbol{\theta})p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$.

3.3.1 Metropolis–Hastings (**chain** `type mh`)

Metropolis–Hastings (MH) is the classical MCMC algorithm: at each step a candidate parameter vector is drawn from a proposal distribution centered on the current point, the candidate is accepted with probability $\min(1, p(\boldsymbol{\theta}_{\rm new}|D)/p(\boldsymbol{\theta}_{\rm old}|D))$, and either the candidate or a repeat of the current point is added to the chain. XSPEC's implementation lets the user choose the proposal shape (Gaussian, Cauchy, or uniform) and the source of its covariance matrix (the fit covariance, an explicit diagonal, or a previously loaded chain).

Strengths. MH is robust and well-understood. Every acceptance step costs one likelihood evaluation, so the per-iteration cost is the cheapest of the four samplers. It handles arbitrarily shaped posteriors and is the only sampler that lets the user supply a fully custom proposal class through the **initpackage** mechanism (see Appendix G).

Weaknesses. The proposal scale and shape must roughly match the posterior or mixing collapses. Acceptance rates well below $\sim 20\%$ or well above $\sim 60\%$ indicate the scale is off. For highly correlated parameters a Gaussian proposal derived from the fit covariance is essential; a diagonal proposal will accept almost nothing in narrow valleys.

Tips. Always run a fit before the chain so the proposal covariance is drawn from a meaningful Hessian. Examine **chain stat** for each free parameter — the Rubin–Gelman statistic should be below 1.1 and the Geweke statistic should be within ± 2 before the chain is trusted. The fraction-of-repeats line is the easiest way to spot a proposal that is much too wide. When proposals from the Hessian are rejected because the covariance matrix has temporarily-frozen parameters, fall back to `proposal gaussian deltas` <`value`> with <`value`> of order unity.

3.3.2 Goodman–Weare (chain type gw)

The Goodman–Weare (GW) ensemble sampler advances many walkers in parallel by "stretch moves": each walker proposes a new position on the line connecting it to another randomly selected walker. The key property is that GW is invariant under affine transformations of the parameter space, so it does not suffer from anisotropic or correlated posteriors the way MH does. GW is the default **chain type** in XSPEC; in the broader Bayesian community it is the basis of the popular `emcee` sampler.

Strengths. GW is essentially tuning-free. The user chooses only the number of walkers (typically $2D$ to $4D$ where D is the number of free parameters) and the length of the chain; there is no proposal scale to set. It handles correlated and moderately skewed posteriors much better than MH for the same elapsed-time budget. The walker updates are independent within each half-step, so XSPEC can fan them out across multiple processes via **parallel walkers**.

Weaknesses. GW relies on the ensemble spanning the posterior. If walkers are initialized in the wrong basin the algorithm can get stuck visiting only that basin; a strongly multimodal posterior is the worst case. The per-step cost is one likelihood per walker, so GW with a few dozen walkers does many more likelihood evaluations per step than MH; the per-step elapsed time is correspondingly higher.

Tips. Initialize the walkers by fitting first so the covariance-based initialization is used; the alternative (uniform initialization with width ten times the parameter **newpar** delta) is wasteful unless you genuinely have no starting point. With `parallel walkers` > 1, the walkers parameter should divide evenly by the number of parallel worker processes for best load balance.

3.3.3 Hamiltonian Monte Carlo (hmc)

Hamiltonian Monte Carlo (HMC) treats the negative log posterior as a potential energy, augments the parameter space with auxiliary momenta, and integrates Hamilton's equations to propose long-range moves that are accepted with very high probability. XSPEC's implementation uses the No-U-Turn Sampler (NUTS) variant, which removes the need to tune the integration length, together with a warm-up phase that adapts the step size and a diagonal mass matrix.

HMC requires the gradient of the log posterior with respect to every free parameter. XSPEC computes this analytically for models that have registered gradient implementations (the majority of additive, multiplicative, and convo-

lution models that ship with XSPEC) and falls back to finite differences for the remainder. Some classes of models — notably the OGIP table models and several legacy Fortran routines — still use the finite-difference path.

Strengths. HMC scales much better than MH or GW into high dimensions, where the latter's per-step efficiency falls exponentially. On smooth posteriors with $D \gtrsim 10$ parameters HMC often produces an effective sample per few likelihood evaluations, whereas MH may need hundreds. Multiple chains can be run in parallel via **parallel hmc**; the implementation forks chains rather than walkers, so the speedup is close to linear with cores until the chains start sharing memory bandwidth.

Weaknesses. The per-leapfrog-step cost is one gradient evaluation, which for analytically supported models costs about as much as one likelihood, but for finite-difference fallbacks costs $D+1$ likelihoods. A single NUTS step takes 8–32 leapfrog steps on typical X-ray spectra, so HMC's per-sample cost can be 100–1000 times that of MH if many parameters fall back to finite differences. HMC is also unforgiving of pathological geometry: posteriors with funnels, ridges, or near-zero-volume modes will produce divergences that show up in the output as warnings.

Tips. Warm up generously; a thousand warm-up samples (**hmc warmup**) is a sensible default. Watch the per-chain divergence count printed at the end of the run; if it exceeds a few percent of the total samples the step size adapted too aggressively and the parameterization should be reconsidered. Use **hmc** `target_accept 0.9` if divergences persist (the default is 0.8). For multimodal posteriors HMC will get trapped in whichever mode the warm-up converged into; this is by design and must be addressed with a different sampler or with explicit tempering.

3.3.4 Nested sampling (nest)

Nested sampling, introduced by Skilling (2004), is fundamentally different from the three MCMC samplers above. Rather than drawing from the posterior directly, it maintains a set of K "live points" distributed according to the prior and iteratively replaces the lowest-likelihood live point with a new draw from the prior, conditioned on having a higher likelihood than the one removed. As the algorithm proceeds the live points climb up the likelihood surface and converge on the posterior modes. The posterior samples are recovered as a weighted re-sampling of the removed ("dead") points; the evidence

$$Z = \int p(D|\boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta} \tag{3.3}$$

falls out for free as a side product.

Strengths. Nested sampling is the natural choice when the goal includes model comparison via Bayes factors. It explores multi-modal posteriors much more reliably than any MCMC method because the live points span the entire prior at the start. It does not require a starting fit; running **fit** before **nest run** has no effect on the sampler at all. The XSPEC implementation supports multi-process parallelization via **parallel nest**, which uses the K-point-removal generalization of Buchner (2014) to remove K live points per iteration and parallelize the K resamplings.

Weaknesses. The per-iteration cost is one or more constrained prior samples, which can themselves require many likelihood evaluations. Nested sampling is slower than HMC on smooth unimodal posteriors but much more reliable on hard ones. The number of iterations scales as roughly $H \cdot K \cdot D$ where H is the information gained in going from the prior to the posterior — the Kullback–Leibler divergence between the two distributions, measured in nats (the unit of information based on the natural logarithm, as the bit is based on log base 2; roughly, H counts how many e-foldings the posterior volume is compressed relative to the prior volume). For typical X-ray problems this is a few thousand iterations and $K = 400$ live points is reasonable.

The strongest constraint nested sampling places on the user is that the prior must be properly normalized on a bounded domain. Parameters with default flat priors over many decades will cause **nest run** to refuse to start; the user must apply `jeffreys` (with the parameter's hard lower limit pushed above zero by **newpar**) or another integrable prior before invoking nest.

Tips. Set **nest live** 400 as a default; lower values risk under-exploring multi-modal regions, higher values cost proportionally more. Use **parallel nest** N where N is roughly the number of physical cores; the speedup is sub-linear but significant. Allow an elapsed-time budget of order an hour for $\sim$6 parameter problems on a single CCD; multi-instrument joint fits with response convolution dominate by an order of magnitude.

3.4 A worked example: a simulated Chandra spectrum

To make the trade-offs concrete, we walk through the same fit with all four samplers on a single inexpensive problem. The data is a **fakeit** Chandra ACIS-S spectrum (RMF and ARF taken from cycle 16, 512 channels in the 0.5–8 keV band) with a 10 ks exposure, fit to `TBabs*apec` with three thawed parameters (`nH`, `kT`, `norm`; abundance held at solar). Numbers given below are from a single dual-cycle run of the XSPEC benchmark harness on a four-core Apple

Silicon host; absolute elapsed times will differ on other hardware but the relative ordering is robust.

3.4.1 Setting up the data and finding the MLE

A minimal setup script. The **fakeit** step builds the simulated spectrum; in practice you would substitute a real spectrum here.

```
statistic cstat
abund angr
xsect vern
xset delta 0.01

model TBabs*apec & 0.05 & 1.0 & 1.0 & 0.0 -1 & 0.04 &
fakeit none & aciss_aimpt_cy16.rmf & aciss_aimpt_cy16.arf
       & y & & fake.fak & 10000.0 & 1.0 & 10000.0 &

data 1:1 fake.fak
ignore **-0.5 8.0-**

method leven 1000 0.005
fit
fit
```

The two consecutive **fit** calls let XSPEC refresh the covariance matrix after any parameter that was temporarily frozen during the first pass thaws back – the second fit converges in zero iterations and updates the Hessian, which the samplers below will use as their warm-start proposal.

3.4.2 Running each sampler

We run each of the four samplers with the same total budget and parallel mode (**parallel** `walkers 4` for the ensemble methods, **parallel** `hmc 4` for HMC, and **parallel** `nest 4` for the nested sampler). All commands are issued from the same XSPEC session after the **fit** block above.

Metropolis–Hastings. Two proposal commands are issued so that the Hessian-based proposal wins when accepted, with the `deltas` form as a fallback in case the Hessian is degenerate (rare for a 3–D well-constrained problem).

```
chain proposal gaussian deltas 0.5
chain proposal gaussian fit
chain type mh
chain length 10000
chain burn 1000
parallel walkers 4
chain run mh.fits
```

Observed: 45.6% acceptance, 4 walkers each producing 2500 post-burn samples, total elapsed time 58 s.

Goodman–Weare. Same Hessian-based proposal; the walker ensemble (10 by default, raised to 32 in the example below) takes care of step scaling via the stretch move, so the proposal width matters less here than for MH.

```
chain proposal gaussian fit
chain type gw
chain length 5000
chain burn 1000
chain walkers 32
parallel walkers 4
chain run gw.fits
```

Observed: 9 s elapsed time (the 32-walker ensemble parallelizes efficiently across 4 cores).

Hamiltonian Monte Carlo. No proposal command – HMC derives its trajectories from gradients of the log-posterior. We use a deliberately small budget here to keep the example short; for a production run aim for at least 1000 warmup + 1000 samples per chain.

```
hmc warmup 100
hmc samples 100
hmc chains 4
hmc target_accept 0.8
parallel hmc 4
hmc run hmc.fits
```

Observed: 590 s elapsed time. At this small budget HMC has not yet built up a good ESS; the same model at the documented default budget (1000+1000) finishes in roughly 1.5–2 hours elapsed time and converges cleanly.

Nested sampling. This is the only sampler that does not warm-start from the MLE; it samples from the prior and uses the likelihood to compress towards the posterior. Two of our parameters (`nH` and the apec `norm`) have hard-coded upper bounds that span far more than four decades, so we narrow their search ranges and switch to a Jeffreys (log-uniform) prior before invoking **nest**.

```
newpar 1 0.05 0.001 1e-6 1e-6 100000 1e6
newpar 5 0.04 0.01  1e-30 1e-30 1e20 1e24
bayes on
bayes 1 JEFFREYS
bayes 5 JEFFREYS

nest live 400
parallel nest 4
nest run nest.fits
```

Observed: 155 s elapsed time, evidence $\log Z = -236.1 \pm 0.2$.

3.4.3 Comparing the results

sampler	elapsed (s)	Rhat	min-ESS	ESS/sec	converged
MH	58.5	1.001	796	13.6	yes
GW	9.1	1.364	78	8.5	no
HMC	590.7	1.056	54	0.1	marginal
nest	155.0	n/a	3765	24.3	yes

Convergence here means Rhat < 1.01 (for multi-chain runs) and min-ESS ≥ 400. Rhat is undefined for nested sampling because it produces a single weighted sample stream.

Three things to notice:

- **Nested sampling and Metropolis–Hastings both converge, for very different reasons.** MH wins on per-sample cost (the chain is essentially a sequence of single forward evaluations each) and the Hessian-proposal lands it inside the textbook 25–50% acceptance window on a 3–D Gaussian-shaped posterior. Nested sampling wins on robustness – it returns the same posterior regardless of where the chain started, and it also gives us $\log Z$ for free, which the others do not.
- **Goodman–Weare looks fast but did not converge.** The 9 s elapsed time is misleading: the 32-walker ensemble produced ESS 78 across 4992

samples, so the effective rate is poor. GW typically needs roughly 30 × the chain length of MH to reach the same ESS because the walkers spend many iterations exploring correlations within the ensemble before contracting onto the posterior. Raising **chain** `length` to 30000 (about 55 s elapsed time) brings min-ESS into the same range as MH.

- **HMC was budget-starved, not failing.** The Rhat of 1.06 across 4 chains tells us the chains have not fully mixed; the per-sample cost is high because each NUTS trajectory requires roughly $2^{\text{tree_depth}}$ gradient evaluations (`tree_depth` $\sim$ 8 for a well-warmed-up chain). At the documented default (1000 warmup + 1000 samples) the same Chandra problem finishes in roughly 1.5–2 hours elapsed time and the Rhat drops below 1.01. HMC's payoff appears at higher dimensionality: it is the only sampler in this list whose per-sample cost grows only mildly with the number of thawed parameters, because the gradient computation is independent of parameter count once you are using the analytic-VJP pipeline (see Appendix H.1).

3.4.4 When to pick what, for this kind of problem

For a 3–6 parameter spectrum with a well-behaved posterior, like this one, the practical recommendation is:

- **First fit:** use **nest** with **parallel** `nest 4` as the first-pass exploration. It gives you both the posterior and the evidence; the elapsed time is tolerable for everything up to about ten parameters with a moderate-binning response. (Goodman–Weare is the default sampler and is the lowest-friction choice for routine work — see the decision flow below — but on a short benchmark like this one its effective-sample yield per unit time is poor.)

- **Refinement around a known mode:** switch to **chain** `type mh` with the **chain** `proposal gaussian fit` proposal. It is cheap, converges quickly, and produces independent-looking samples without further tuning.

- **Higher-dimensional problems** (more than a dozen parameters), or models where the posterior has long correlated directions: switch to **hmc**, accept the higher per-sample cost, and budget for at least 1000 warmup samples per chain.

- **Avoid Goodman–Weare for small-budget runs.** It is excellent on long runs at high dimension where it can amortize the walker burn-in, but for short benchmarks like this one its ESS per sample is too low.

3.5 Choosing a sampler

The following table summarizes the trade-offs between the four samplers.

	MH	GW	HMC	nest
Tuning effort	Moderate	Low	Moderate	Low
Per-sample cost	Cheapest	Low	High	Highest
Effective samples per unit time, smooth posterior	Low to moderate	Moderate	High	Moderate
Robustness to multimodality	Poor	Poor	Poor	Good
Robustness to correlated parameters	Poor (without good covariance)	Good	Excellent (with gradients)	Good
Returns evidence Z?	No	No	No	Yes
Requires gradients?	No	No	Yes	No
Warm-start from MLE?	Helpful	Helpful	Essential	Ignored

A useful decision flow:

- Few parameters ($D \leq 5$), smooth posterior, single source — start with GW (the default). It is the lowest-friction option for routine work.
- Moderate to many parameters ($D \geq 10$), gradients available for the model components in use — HMC. The gain over GW becomes decisive past about $D = 15$.
- Multi-modal posterior, or a model-comparison question that needs Bayes factors — nested sampling. Accept the higher elapsed time as the cost of reliability and the bonus of Z.
- Custom proposal needed (e.g. exploiting an exact conditional) or working with a model class that defeats both HMC and nest — MH with a user-defined proposal class through **initpackage**.

3.6 Model comparison by evidence

The samplers above answer "given this model, what are the parameters?" A distinct question is "which of several competing models does the data prefer?" The Bayesian answer is the *evidence*, the marginal likelihood already introduced

as the by-product of nested sampling,

$$Z = p(D|M) = \int p(D|\boldsymbol{\theta}, M)\, p(\boldsymbol{\theta}|M)\, d\boldsymbol{\theta}. \tag{3.4}$$

For two models the ratio of evidences is the *Bayes factor* $B_{12} = Z_1/Z_2$, and by Bayes' theorem the posterior odds are the Bayes factor times the prior odds,

$$\frac{p(M_1|D)}{p(M_2|D)} = B_{12}\,\frac{p(M_1)}{p(M_2)}. \tag{3.5}$$

With n models and equal prior probabilities the posterior model probabilities are simply $P(M_i|D) = Z_i/\sum_j Z_j$.

The evidence differs from a maximum-likelihood comparison in a crucial way: it integrates the likelihood over the whole prior rather than evaluating it at the single best-fit point. A model with extra parameters can reach a higher peak likelihood, but if those parameters spread the prior mass over regions the data rule out, the integral is penalized. This built-in Occam factor is exactly the prior volume the extra freedom occupies — a continuous, prior-aware version of the parameter-count penalties used by the information criteria AIC and BIC (indeed BIC $\approx -2\ln Z$ in the large-data limit). Because the penalty *is* the prior volume, the evidence depends on the prior in a way that posterior parameter estimates usually do not: a needlessly wide prior on a parameter inflates the Occam penalty and can wrongly disfavor a model. Setting meaningful priors is therefore part of the comparison, not an afterthought, and is the reason **nest** (and hence the evidence-based comparison below) refuses to run under unbounded multi-decade flat priors.

The customary scale for interpreting a Bayes factor is that of Kass & Raftery (1995), stated in terms of twice the log-evidence difference $2\,\Delta\ln Z = 2\ln B$: a value above 2 is "positive" evidence, above 6 "strong", and above 10 "decisive" for the better model. A difference of order the $\log Z$ uncertainty reported by the sampler is not significant.

In XSPEC this is automated by **compare** `nest`. With **bayes** `on` and priors set, the command runs nested sampling on the current model and each supplied alternative, then tabulates $\log Z$, $\Delta\ln Z$, the Kass–Raftery quantity $2\,\Delta\ln Z$, and $P(M|D)$, ranking by evidence rather than by the AIC/BIC of the non-Bayesian **compare** modes. Priors for each alternative are taken from the current model by parameter name, or are read from a saved-model candidate supplied as `@`<file.xcm> when a model needs priors of its own; see the **compare** command description in Chapter 6 for the full syntax and a worked example. The cost is one full nested-sampling run per model, so the same **nest live** and **parallel nest** considerations apply as for a single **nest** run.

3.7 Convergence diagnostics

No sampler is self-diagnosing; the user must check that the chains have converged before trusting credible intervals.

For the chain-based samplers (MH and GW), the standard **chain stat** output reports per-parameter mean, variance, Rubin–Gelman convergence statistic (across multiple chains, if present), Geweke convergence statistic (within a single chain), and the fraction of repeated steps. The classical Rubin–Gelman $\hat{R}$ should be below 1.1 (a looser rule of thumb than the rank-normalized Rhat introduced below); the Geweke statistic should be within ± 2. A fraction of repeats above about 0.8 indicates the proposal scale is too large and the chain has effectively stalled.

For a fuller, rank-normalized assessment, **chain diag** prints the diagnostic suite of Vehtari et al. (2021): for every parameter it reports the classical and rank-normalized split Rhat, the bulk and tail effective sample sizes, the integrated autocorrelation time and the ratio N/τ, and — for Goodman–Weare — a sticky-walker check, opening with a one-line `[ OK ]`/`[WARN]`/`[SHORT]` verdict. Its thresholds are tighter than the classical rule of thumb above: rank-normalized Rhat < 1.01, bulk- and tail-ESS ≥ 400, and at least 50 samples per integrated autocorrelation time. The same report is printed automatically at the end of a **chain run** and its verdict is appended to **chain info**, so the convergence assessment is normally already in front of you when a run finishes; see the **chain** command entry for the full description.

For HMC, the implementation prints a final summary line per chain giving the number of divergences, the maximum tree depth reached, and the energy-conservation error. Divergences should be rare ($\lesssim 1\%$); persistent divergences indicate the geometry of the posterior is incompatible with the diagonal-mass-matrix assumption, and a non-centered reparameterization may help.

For nested sampling, the algorithm terminates automatically when the remaining prior volume can contribute at most a user-controlled fraction of the evidence. The output reports the final $\log Z$ together with its estimated uncertainty, and the posterior sample is written along with importance weights so the user can verify that no single sample dominates the effective sample size.

In addition to the built-in diagnostics, the chain files produced by all four samplers are written in a FITS format compatible with external analysis tools (notably `arviz` and `corner.py` in Python). Loading the chain into one of those packages complements the built-in **chain diag** report and is especially useful for visualizing the joint posterior structure.

3.8 Common pitfalls

- **Forgetting `bayes on`.** Setting prior expressions with **bayes** <par> <type> declares them, but the samplers do not consult them until inference is switched on.
- **Improper priors with nested sampling.** A flat prior on a parameter that ranges over many decades produces a prior with effectively no normalization. **nest run** will refuse to start; the fix is to set `jeffreys` on the parameter and push the hard lower limit above zero with **newpar**.
- **Stale fit covariance.** For MH the proposal covariance is taken from the current fit. If the fit converged to a basin far from the posterior mode of interest — for example because of "temporarily frozen due to zero norm" parameters — the proposal will reject almost everything. Refit, or supply a diagonal proposal with `proposal gaussian deltas`.
- **Running HMC without warm-up.** The step size and mass matrix must adapt before the production samples are taken; running with **hmc warmup** $\ll 200$ on a non-trivial model guarantees divergences. The default of 1000 warm-up iterations is sensible for most problems.
- **Comparing chains of unequal length.** Rubin–Gelman assumes the chains being compared have run for the same number of iterations. Pad the shorter chain or discard the excess from the longer one before invoking the statistic.
- **Reporting frequentist confidence intervals from Bayesian samples.** Posterior credible intervals are not frequentist confidence intervals, and they should not be reported as such. Be explicit in the text about which kind of interval is quoted.

3.9 Further reading

The standard references for Bayesian methods in physical-science data analysis are Gregory (2005) and Sivia and Skilling (2006). For the specific algorithms used here: Metropolis–Hastings is described in any modern MCMC textbook; Goodman–Weare is the basis of Foreman-Mackey et al. (2013); the NUTS variant of HMC is Hoffman and Gelman (2014); nested sampling is Skilling (2004) with the multi-process extension of Buchner (2014). The interpretation of Bayes factors and the evidence scale used by **compare** `nest` follow Kass and Raftery (1995). The discussion of convergence diagnostics draws on Gelman and Rubin (1992) and Geweke (1992).

The result of all this manipulation is shown proudly in Figure 5.10.

5.8 Markov Chain Monte Carlo Example

To illustrate MCMC methods we will use the same data as the first walkthrough.

```
XSPEC> data s54405
...
XSPEC> model phabs(pow)
...
XSPEC> renorm
...
XSPEC> chain type gw
XSPEC> chain proposal gauss deltas 100.0
XSPEC> chain walkers 20
XSPEC> chain length 200000
XSPEC> parallel walkers 10
```

We use the Goodman-Weare algorithm with 20 walkers and a total length of 200,000. For the G-W algorithm the actual number of steps are 200,000/20 but we combine the results from all 20 walkers into a single output file. We start the chain with a set of walkers drawn from a multidimensional gaussian whose sigmas are one hundred times the parameter deltas and whose means are the current model parameters. Note that we used the **renorm** command to make sure that the model and the data have the same normalization. If we had multiple additive parameters with their own norms then a good starting place would be to use the **fit 1** command to initially set the normalizations to something sensible.

```
XSPEC> chain run test1.fits
```

The first thing to check is what happened to the fit statistic during the run. The thin N option on the chain plot means that only every Nth point is plotted. It is important to pick an N which is incommensurate with the number of walkers to make sure that they are sampled equally.

```
XSPEC> iplot chain thin 31 0
PLT> log y
PLT> pl
```

The result is shown in Figure 5.11, which plots the statistic value against the chain step. It is clear that there is an initial burn-in phase before the chain

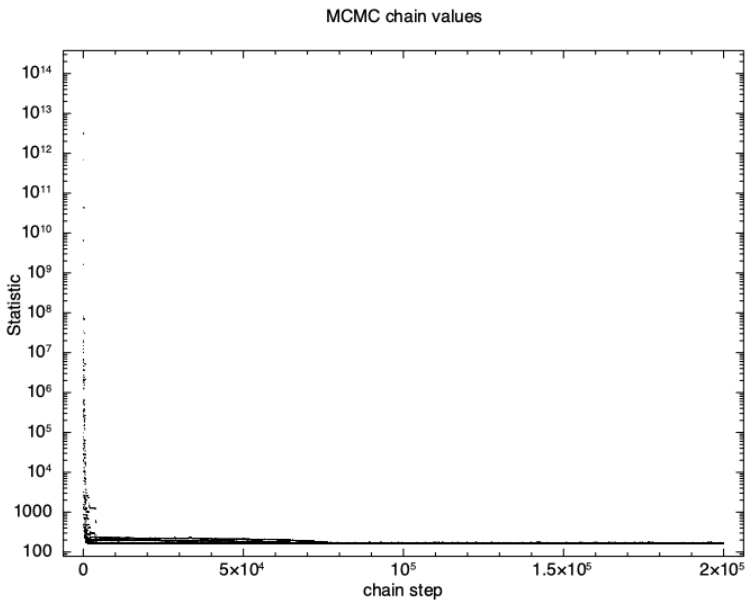


Figure 5.11: The statistic from an MCMC run including the initial burn-in phase.

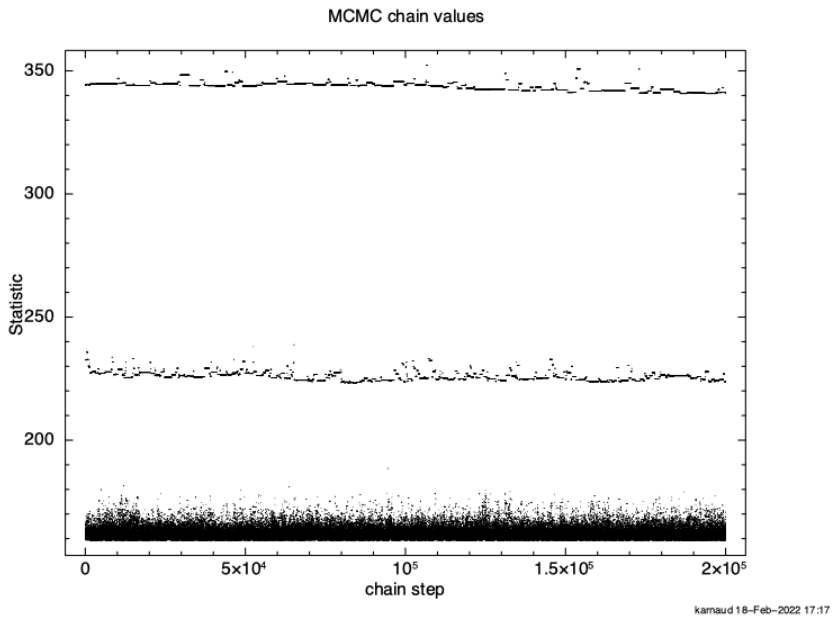


Figure 5.12: The statistic from a 200,000 step MCMC run with a 100,000 step burn in.

reaches equilibrium so we tell XSPEC to discard the first 100,000 steps and then record the next 200,000.

```
XSPEC> chain burn 100000
XSPEC> chain run test1.fits
```

The result is shown in Figure 5.12, which again plots statistic value against the chain step. What we see is that some of the walkers have not mixed, something which happens sometimes. Finally, we run a chain with ten times as many steps and ten times as long a burn-in to give Figure 5.13. The parameter values can be plotted either singly using e.g. **plot chain 2** for the power-law index or in pairs eg **plot chain 1 2** giving a scatter plot as shown in Figure 5.14.

Using the **error** command at this point will generate errors based on the chain values.

```
XSPEC>error 1 2 3
Errors calculated from chains
```

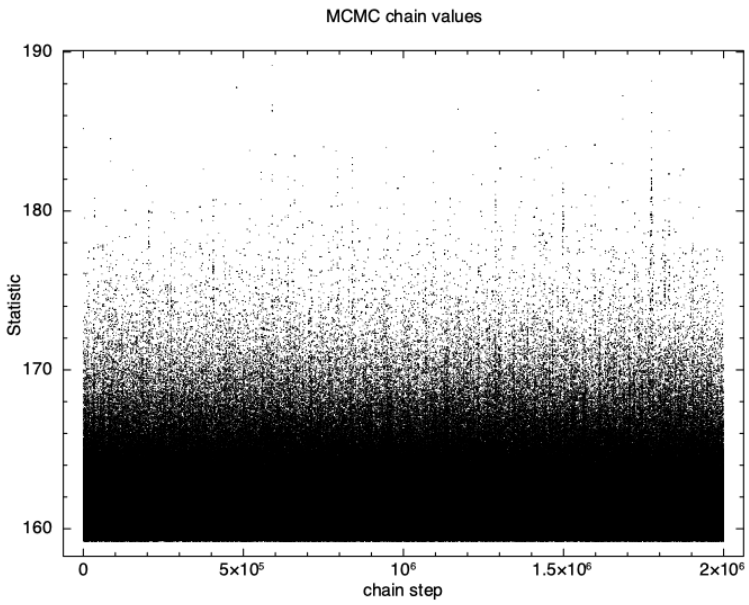


Figure 5.13: The statistic from a 2,000,000 step MCMC run with a 1,000,000 step burn in.

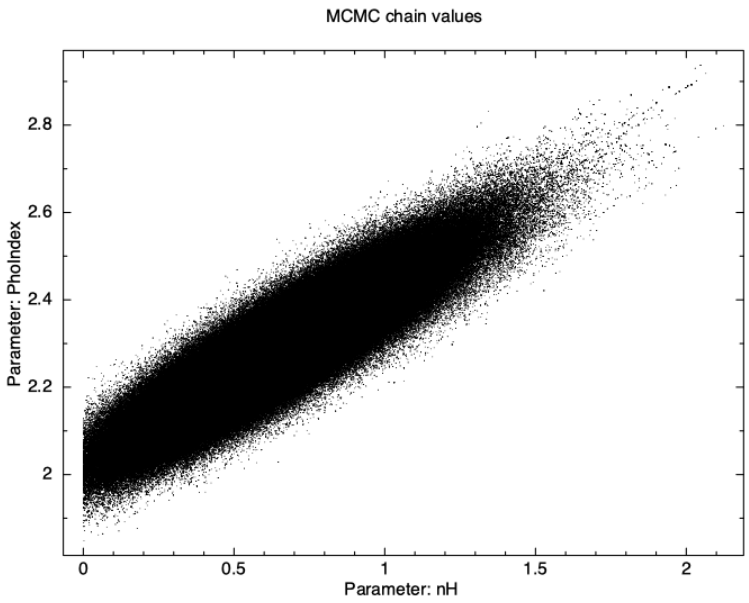


Figure 5.14: The scatter plot from a 200,000 step MCMC run.

```
Parameter   Credible Interval (90%)
    1      0.202032      1.12274
    2       2.08515      2.49323
    3     0.0102687     0.019415
```

The 90% confidence ranges are determined by ordering the parameter values in the chain then finding the center 90%.

To make confidence regions in two dimensions the **margin** command takes the same arguments as **steppar**.

```
XSPEC>margin 1 0.0 2.0 25 2 1.8 2.9 25
```

Then use **plot integprob** to produce a contour plot of the integrated posterior probability where the contours enclose 68, 95, and 99.7% of the probability (see Figure 5.15).

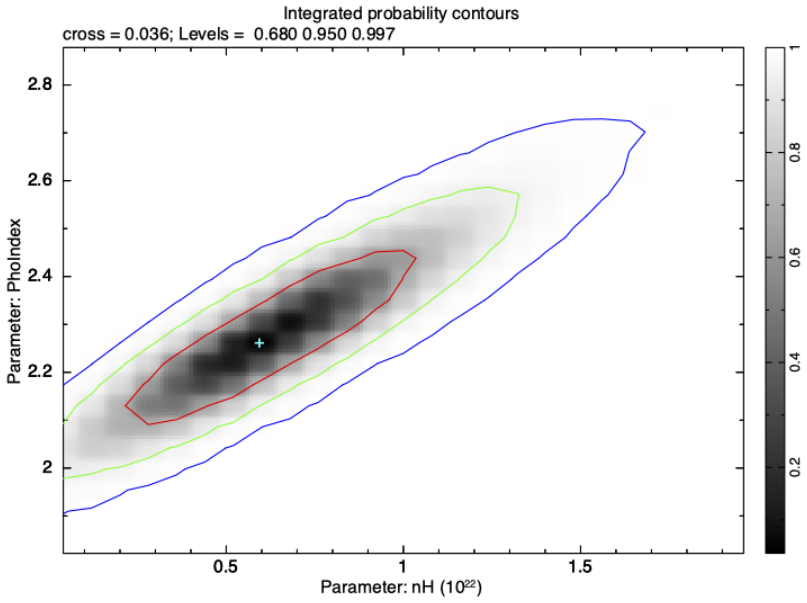


Figure 5.15: The integrated probability plot from a 200,000 step MCMC run.

5.9 Spectropolarimetry Example

XSPEC can fit spectropolarimetric models to spectra for the I, Q, and U Stokes parameters. The following is a toy example for IXPE data. The responses are notional and should not be used with actual data.

The example below uses the phenomenological polarization models (`polconst`, and similarly `pollin` and `polpow`) applied as a multiplicative factor to a spectral model. XSPEC also provides physical additive reflection models, `stdisc`, `sttorus`, `stcone` and `stbowl`, which return the I, Q, and U Stokes spectra directly: with their `Stokes` selector parameter set to -1 they dispatch on the same `Stokes` XFLT keyword and so are fit against the I, Q, U triplet in exactly the same way.

```
XSPEC> data 1:1 toy_point_source_du1_pha1.fits
XSPEC> data 1:2 toy_point_source_du1_pha1q.fits
XSPEC> data 1:3 toy_point_source_du1_pha1u.fits
XSPEC> data 2:4 toy_point_source_du2_pha1.fits
XSPEC> data 2:5 toy_point_source_du2_pha1u.fits
XSPEC> data 2:6 toy_point_source_du2_pha1q.fits
XSPEC> ignore 1-6:0.0-2.0 8.0-**
```

This has read in the I, Q, U triplet for two of the three telescopes and placed each triplet in its own data group. The I, Q, and U spectra are distinguished by having different values of their XFLT0001 keyword: 'Stokes:0', 'Stokes:1', and 'Stokes:2' for I, Q, and U respectively.

```
XSPEC>model polconst*powerlaw
...
========================================================================
Model polconst<1>*powerlaw<2> Source No.: 1   Active/On
Model Model Component  Parameter  Unit     Value
par  comp
                         Data group: 1
1    1   polconst   A                    1.00000      +/-  0.0
2    1   polconst   psi        deg       45.0000      +/-  0.0
3    2   powerlaw   PhoIndex             1.00000      +/-  0.0
4    2   powerlaw   norm                 1.00000      +/-  0.0
                         Data group: 2
5    1   polconst   A                    1.00000      = p1
6    1   polconst   psi        deg       45.0000      = p2
7    2   powerlaw   PhoIndex             1.00000      = p3
8    2   powerlaw   norm                 1.00000      = p4
------------------------------------------------------------------------
```

defines a power-law model with constant polarization fraction and angle.

to the name and a `.log` suffix added. To change the chattiness level for the log file (ie. the amount of information written to the log file) use the **chatter** command. The default chatter level for the log file is 10.

Examples:

```
XSPEC> log
//Turn on the log file (default xspec.log).
XSPEC> log none
//Close the log file.
XSPEC> log >mylog
//Append to the log file (mylog.log)
XSPEC> chatter ,, 12
//Set the log file chattiness to 12.
```

6.3.6 parallel

enable parallel processing for particular tasks in XSPEC.

Syntax: **parallel** <task> <max num of processes>

where <task> is currently limited to `leven`, `global`, `error`, `steppar`, `goodness`, `walkers`, `hmc`, `nest`, or `sim`. For best results, it is recommended that you set <max num of processes> to the number of CPU cores on your machine. Set <max num processes> back to 1 to turn parallel processing off for the particular task. To display current settings, type 'parallel' with no arguments.

The `leven` option will spawn up to <max num> processes during the Levenberg-Marquardt fitting, specifically to perform the N independent calculations of the parameter first-order partial derivatives (N being the number of variable fit parameters).

The speed-up that one can expect is highly dependent upon the model in use. For simpler models with quick calculation times, you will probably see little to no speed gain with **parallel leven**. But with multi-core CPUs, gains should be quite noticeable when the model calculation consumes a large fraction of the overall fitting time. For example, with fits using the time-intensive **sedov** model on a 4-core machine, we've typically seen about a 40% reduction in fit time compared with the single processing case.

The `error` option is for running parallel computations within XSPEC's **error** command. This enables the error calculations for multiple parameters to be

performed simultaneously. The speed-up here should simply be proportional to the number of cores available. However for cases where complications are reported (such as a new minimum found, or a non-monotonicity in the statistic space), it is recommended that you perform the error calculations in standard single-process mode.

When the `steppar` option is set, XSPEC will divide the N-dimensional steppar grid into <`max num`> sections of equal size, and spawn a separate process for calculating each section.

If both **parallel** `leven` and `error` or `steppar` are in use, XSPEC will temporarily disable the lower-level `leven` parallelization when running the higher-level parallel `error` or `steppar` command calculations.

The `goodness` option is for parallelizing the calculations of the simulated spectra during a **goodness** command run.

The `walkers` option may be used to speed up the calculation of sets of `walkers` needed for Monte Carlo Markov Chain runs using the Goodman-Weare algorithm (see **chain**).

The `hmc` option controls concurrency for the **hmc** command. Each **hmc** chain is run in its own sub-process when **parallel hmc** <`N`> is set to $N \geq 2$; with the default `1` all chains are run back-to-back in the parent process, which is fine for production sampling but does not give an honest Rubin–Gelman $\hat{R}$ because the chains are seeded sequentially from a single RNG stream. Set `hmc` to the number of **hmc** `chains` for parallel sampling with independent seeds.

The `nest` option enables a worker pool for the **nest** command. With **parallel nest** <`N`> set to $N \geq 2$, the nested sampler dispatches the likelihood-restricted prior draws across N subprocesses; speed-up is sub-linear (the ellipsoid update is serial) but typically a factor of 2–3 on a four-core host.

The `global` option spreads the Differential Evolution search of **fit global** (and **improve**) across <`N`> subprocesses, which evaluate the statistic for the trial population in parallel. As with `leven`, set it to roughly the number of physical cores; the global-search result is unchanged (bit-identical and reproducible under **xset seed**), only faster.

The `sim` option distributes the realizations of the **sim** command across <`N`> subprocesses. The parameter sets are drawn in the parent process before the work is dispatched, so the results are identical (and reproducible under **xset seed**) whether the run is serial or parallel; only the per-realization counting noise differs.

Examples:

```
XSPEC> model cflow
// Using a model with 5 variable fit parameters.

XSPEC> parallel leven 4
XSPEC> fit
// Calculations for the 5 parameters will be divided amongst
// 4 processes during the fit.

XSPEC> parallel leven 1
 // Restores single-process calculation to the
 // Levenberg-Marquardt algorithm.

XSPEC> parallel error 3
 // Allow up to 3 simultaneous 'error' parameter calculations
 // to be performed in parallel.

XSPEC> error 2 3 6
 // Perform error calculations on parameters 2, 3, and 6 in parallel.

XSPEC> parallel steppar 4
      // The following 20x30 steppar grid will be split amongst
      // 4 parallel processes.
XSPEC> steppar 1 10. 11. 20 2 .5 .8 30

 // Display current settings:
XSPEC> parallel
Maximum number of parallel processes:
   error: 3
   global: 1
   goodness: 1
   hmc: 1
   leven: 1
   nest: 1
   sim: 1
   steppar: 4
   walkers: 1
```

6.3.7 query

set a default answer for prompts in scripts

Switch on/off the continue fitting questions.

Syntax: **query** <option>

```
XSPEC> response 2:2 none
// The second source is now removed from spectrum 2.
```

6.5 Fit Commands

6.5.1 bayes

set up for Bayesian inference

Syntax:	**bayes**	<option>
	bayes	<mod par #> <prior type> [<hyperparameters>]
	bayes	smooth <par> [order 1|2] [space lin|log] [lambda <value> | lcurve | discrep] [radii $<r_1>$ ... $<r_N>$]
	bayes	smooth clear

where <option> ::= [off |on |cons]. If a parameter number is given as the first argument then this command sets up the prior for the specified model parameter but does not turn Bayesian inference on. If the first argument to the **bayes** command is not a parameter number then one of the options off, on, or cons is used. The first two turn Bayesian inference off or on, while cons turns Bayesian inference on and gives all parameters a constant prior. The options for prior types are as follows:

Prior type	Log(prior)
cons	0
exp	$-p/h_1 - \ln(h_1)$
jeffreys	$-\ln(p)$
gauss	$-0.5\ln(2\pi h_2) - 0.5(h_1-p)^2/h_2^2$
lognormal	$-\ln(p\sqrt{2\pi}h_2) - 0.5(\ln(p)-\ln(h_1))^2/h_2^2$
gamma	$h_1\ln(h_2) + (h_1-1)\ln p - h_2 p - \ln\Gamma(h_1)$
cauchy	$-\ln(1+(p-h_1)^2/h_2^2) - \ln\pi - \ln h_2$
beta	$(h_1-1)\ln x + (h_2-1)\ln(1-x) - \ln\Gamma(h_1+h_2) - \ln\Gamma(h_1) - \ln\Gamma(h_2)$
studentst	$\ln\Gamma((h_1+1)/2) - 0.5\ln(h_1\pi) - \ln\Gamma(h_1/2) - ((h_1+2)/2)\ln(1+p^2/h_1)$
halfnormal	$-0.5\ln(h_1^2\pi/2) - p^2/(2h_1^2)$
powerlaw	$-h_1\ln(p) - \ln Z$ for $h_2 \le p \le h_3$, where the prior density is p^{-h_1}/Z and $Z = (h_3^{1-h_1} - h_2^{1-h_1})/(1-h_1)$, or $\ln(h_3/h_2)$ when $h_1 = 1$
isotropic	$\ln(\sin(p)) + \ln(\pi/360)$, with p in degrees over $[0, 180]$
shiftedlognormal	$-\ln((p-h_1)\sqrt{2\pi}h_3) - 0.5(\ln(p-h_1) - \ln(h_2))^2/h_3^2$

Where p is the parameter value, $h_\#$ the hyperparameter values, and for the beta prior x = *(p-min)/(max-min)* where *min* and *max* are the parameter hard limits.

Multi-parameter smoothness prior. The priors above act on one parameter at a time. The **bayes** `smooth` subcommand instead applies a single prior that *couples* a chosen model parameter <par> (its local index within the model) across all datagroups, penalising shell-to-shell roughness of that parameter. It is the in-engine regularizer for **projct** de-projection, where the per-datagroup normalisations (or temperatures) recovered by inverting the projection are noisy and anti-correlated; see the **projct** model description for the underlying failure modes. The prior is a correlated Gaussian

$$\ln p = -\tfrac{1}{2}\,\lambda\,\|L\,q\|^2, \qquad q_k = p_k \ (\texttt{lin}) \ \text{ or } \ \ln p_k \ (\texttt{log}),$$

where the sum runs over the datagroups, L is a spacing-aware finite-difference operator built from the shell radii, and λ is the regularisation strength. The options are:

`order 1|2` `1` penalises the gradient of the profile (biasing towards a flat profile); `2` penalises the curvature (biasing towards a power-law-like profile).

Default 2.

`space lin|log` whether the operator acts on the parameter (`lin`) or its logarithm (`log`). Default `log`, appropriate for normalisations that vary over orders of magnitude.

`lambda <value> | lcurve | discrep` a fixed strength `<value>`; or `lcurve` to select it automatically at the corner of the misfit-versus-roughness L-curve; or `discrep` to select it by the discrepancy principle (increase the smoothing until the data misfit reaches the number of degrees of freedom). The `lcurve` and `discrep` options refit the model over a grid of λ and leave the model fitted at the chosen value.

`radii <r1> ... <rN>` the shell radii, one per datagroup. If omitted they default to the XFLT `major` axis of each datagroup (the **projct** convention), so cluster users need not re-enter them.

The smoothness term is consumed by the same machinery as the per-parameter priors, so it regularises Levenberg–Marquardt fits, the error calculation, and all four samplers (under **nest** it is sampled through a spectral transform of its rank-deficient precision). The members must be free, unlinked, and free of a per-parameter prior. **bayes** `smooth clear` removes all smoothness terms (as does **bayes** `cons`). Registered terms are written to **save** files and listed by **show** `bayes`.

6.5.2 chain

run a Monte Carlo Markov Chain

Syntax: **chain** `[best] [burn <length>] [clear] [diag] [dic] [filetype fits|ascii] [info] [length <length>] [load <filename>] [proposal [<distr> <source>] |[<user-defined>]] [rand on|off] [recalc] [rescale <factor>] [run [>]<filename>] [stat <par num>] [temperature <value>] [type mh|gw] [unload <range>] [walkers <value>]`

If the proposal source is set to use the fit correlation matrix (the default), you must perform a fit before running any chains.

When chains are loaded (and their parameters correspond to the currently loaded model), they will be used by the various XSPEC commands that require distributions of parameter values, such as **eqwidth** or **flux** when calculating

error estimations. The **error** command itself will also use the loaded chains, determining the error range from a central percentage of the sorted chain values. This is likely to be faster than the **error** command's standard algorithm when not using chains.

`best` Finds the parameter and statistic values for the point in the loaded chains with the best statistic value.

`burn` <`length`> Specifies that the first <`length`> steps should be thrown away prior to storing the chain.

`clear` Does a reset and removes all chains from the list.

`diag` Prints the Markov-chain convergence and effective-sample-size diagnostics of Vehtari et al. (2021) for the loaded chains. For each variable parameter the report tabulates the classical split Rhat, the rank-normalized Rhat, the bulk and tail effective sample sizes (ESS), the integrated autocorrelation time τ and the ratio N/τ, and, for Goodman-Weare chains, the per-walker sticky-walker ratio ($\tau_{\max}$/median), together with a 1-D marginal bimodality coefficient. The report opens with a one-line verdict:

- `[ OK ]` if every check is within its threshold;
- `[WARN]` if one or more checks fail (each failing check is then listed);
- `[SHORT]` if the chain is too short (fewer than 8 samples per sub-chain) for the diagnostics to be defined.

The warning thresholds are fixed: rank-normalized Rhat < 1.01; bulk-ESS and tail-ESS ≥ 400; $N \geq 50\,\tau$; and, for Goodman-Weare, a walker τ ratio < 10. The bimodality coefficient is advisory only — a value above $5/9$ suggests a flat-topped or multi-modal marginal and points you at **nest** for rigorous mode separation — and does not by itself raise a `[WARN]`. The full report is also printed automatically at the end of a successful **chain run**, and the one-line verdict is appended to the output of **chain info**.

`dic` Calculates the Deviance Information Criterion and the effective number of parameters for the loaded chains.

`filetype fits|ascii` Chooses the format of the output chain file. `fits` (the default) writes the chain to a binary table in a FITS file. `ascii` writes the chain to a simple text file. Either format is readable when using the **chain load** command.

`info` Prints out information on the current chains, followed by the one-line convergence verdict described under **chain diag**.

`length` <`length`> Sets the length for new chains.

`load` <`filename`> Loads a chain which has been run earlier, stored in file given by <`filename`>.

`proposal` <`distr`> <`source`> Selects the proposal distribution and source of covariance information to be used when running new chains. For the Metropolis-Hastings algorithm this distribution is used at each step of the chain while for the Goodman-Weare algorithm it is only used to set the initial walkers. The default is `proposal gaussian fit`. Currently implemented <`distr`> options are: `gaussian`, `cauchy`, and `uniform`. <`source`> options are:

`chain` Covariance is taken from the currently loaded chains.

`deltas` <`value`> A diagonal covariance matrix is constructed from the parameter deltas multiplied by the value entered on the command line.

`diagonal` <`values`> The values of a diagonal covariance matrix are entered directly on the command line, separated by commas and/or spaces: `C_11 C_22 ... C_nn`.

<`filename`> Covariance is read in from a user-specified text file. The file must contain the values of an NxN matrix where N is the current number of freely varying parameters. The values of each matrix row should be entered on one line with whitespace separation. Since this matrix is always symmetrical, values above the diagonal may be omitted. For example a 2x2 matrix could be entered as:

```
0.98
0.15 0.96
```

`fit` Covariance is taken from the correlation information produced by the current fit.

`limits` Used only for the `uniform` option and draws uniformly from the entire parameter space defined by the parameter hard limits.

`matrix` <`values`> The lower half and diagonal of a symmetrical square covariance matrix are entered directly on the command line, separated by commas and/or spaces: `C_11 C_21 C_22 C_31 C_32 C_33 ... C_nn`

The `uniform` <`distr`> option only accepts <`source`> values of `deltas` or `limits`. **chain proposal** with no other arguments will show a list of all available proposal options. For an alternative to XSPEC's <`distr`> <`source`> proposal options, the user may instead want to provide their own custom randomization algorithm. This can be done by writing their own C++ class(es) derived from an XSPEC randomizer base class. The custom class is added at runtime using the same **initpackage** or **lmod** command sequence as for local models, and is specified by `proposal`

<name> where <name> is the unique name attribute the user provides for their class. Please see Appendix G for more information on writing a custom randomizing class, and **initpackage** for building and loading it.

`rand on|off` Specifies whether the chain start point will be randomized, or taken from the current parameters.

`recalc` A deprecated option that performs the equivalent of `proposal gaussian chain`.

`rescale <factor>` Rescale the covariance matrix used in the proposal distribution by the factor given.

`run [>] <filename>` Runs a new chain written to the specified file, or append to an already loaded file if the '>' character preceeds the filename. The chain is written to the file as it runs so its performance can be monitored by examining the file. For high-chatter settings, additional information is printed to the screen. A long run may be interrupted with Ctrl-C, in which case the chain file will still exist but will not be automatically loaded. If appending to a file, the current `filetype` setting must match the format of the file or XSPEC will prevent it. On successful completion the convergence diagnostics (see **chain diag**) are printed.

`stat [<modName>:]<parIdx>` Writes out statistical information on a particular parameter of the chain, specified by the parameter index number. If the parameter is from a named model then it is indicated by `[<model name>:]<n>`. If the parameter is from a response model then use the format `[<source number>:]r<n>`. If the parameter number is zero then the information is about the fit statistic. The information displayed is:

- The mean of the parameter in each chain.
- If there are multiple chains, the parameter mean over all chains and the variance between chain means.
- The variance over all the chains.
- If there are multiple chains the Rubin-Gelman convergence measure.
- The Geweke convergence measure.
- If using Metropolis-Hastings, the fraction of repeats, defined as the number of lines in the chain file for which all parameter values are identical to the previous line, divided by the number of lines in the file.
- The modern diagnostics of Vehtari et al. (2021) for this parameter (rank-normalized Rhat, bulk- and tail-ESS, integrated autocorrelation time, the bimodality coefficient, and, for Goodman-Weare, the walker τ ratio); see **chain diag**.

`temperature <value>` Sets the temperature parameter used in the Metropolis-Hastings algorithm for the proposal acceptance or rejection. The default value is 1.0 and zero or negative values are forbidden. By using the run append option, it is possible for different sections of the chain file to use different temperatures. The temperatures and the line numbers to which they apply are stored in the header of the FITS format chain files, or in the metadata section at the top of the ASCII text format files.

`type mh|gw` Determines the algorithm used to generate the chain. Choices are `mh` (Metropolis-Hastings) or `gw` (Goodman-Weare, the default). If using Goodman-Weare, must also set the `walkers` parameter. Also note for Goodman-Weare that the calculation of `walkers` sets may be sped up by adjusting the **parallel** command's `walkers` option to perform the calculations over multiple processes. The walkers are initialized in two different ways depending on whether a fit has previously been run. If a fit has been run and a covariance matrix is available then walkers are initialized by drawing from a multi-Normal distribution whose variance matrix is based on the covariance matrix. If a covariance matrix is not available then parameters in each walker are initialized by drawing from a uniform distribution centered on the current fit parameters with width ten times that of the parameter delta. Initial values for walkers can also be generated outside xspec and then imported by writing them to a standard FITS chain output file with number of rows equal to number of walkers. If a chain is now run using the append option (> before the filename) then the starting values for each walker will be taken from the file.

`unload <range>` Removes the chains specified by `<range>` from the list in xspec. Note that this does NOT delete the chain files.

`walkers <value>` Sets the `walkers` parameter for the Goodman-Weare chain algorithm (see **chain type**). This must be an even integer, and both the chain length and burn length should be divisible by it (XSPEC will adjust the lengths to make them so if necessary).

All loaded chains must contain the same fit parameters. xspec will prevent the loading of a chain with a different number of parameters from the currently loaded chains.

Algorithm notes

For the Goodman-Weare (gw) choice the method used to derive new parameter values for the walkers is as follows.

- Divide the walkers into two sets (to allow parallelization)
- For each walker, W, randomly choose a walker, U, from the other set

- Draw a random number Z from the distribution $1/\sqrt{(z)}$, where z lies between 0.5 and 2.0.
- Set the new parameters for W to Z*parameters(W) + (1-Z)*parameters(U)
- Calculate a new fit statistic with these parameters then calculate T = (N-1)*log(Z) - 0.5*(newStat-oldStat) where N is the number of fit parameters
- Draw a random number from a uniform distribution between 0 and 1 and if the ln of this random number is less than T then accept the new parameter values otherwise repeat the old parameter values.

Examples:

```
XSPEC>chain length 100
//Sets length of chains produced by the run command to 100.
XSPEC>chain run chain_file1.out
//Runs a chain based on current valid fit parameters, output to
//chain_file1.out
XSPEC>chain run >chain_file1.out
//Appends another run of length 100 to the end of chain_file1.out
XSPEC>chain load chain_old.out
//Loads a pre-existing chain file, the result of an earlier run
//command.  Warning is issued if not the same length as
//chain_file1.out
XSPEC>chain stat 3
//Prints statistical information on the 3rd parameter of the chain.
XSPEC>chain proposal gaussian myfile.txt
//New chain proposals will be a normal distribution using
//covariance values stored in myfile.txt rather than fit
//correlation matrix.
XSPEC>chain prop gauss diag .1 .001 .0001
// New chain proposals will be a normal distribution using a 3x3
// diagonal covariance matrix with the values from the
// command line.
XSPEC>chain temperature .8
// Sets the Metropolis-Hastings temperature value to .8 for
// future chain runs, replacing the default 1.0.
XSPEC>chain clear
//Removes the 2 loaded chains from xspec's chain list.
```

6.5.3 compare

compare alternative models against the current one

Syntax: **compare** `[global | nest] <alt> [<alt> ...]`

The **compare** command automates model selection. It evaluates one or more user-supplied alternative models against the *current* data, ignore ranges and fit statistic — carrying shared parameters over by name from the current best fit — and tabulates a model-comparison statistic for each, with the current model included as a baseline row. The original model is restored when the command completes (or on any error), so **compare** is non-destructive: it does not change the data, the responses, or the fitted model. A valid **fit** is required before it can be run.

Each <alt> is one of:

{**<model-expr>**} an inline model expression, exactly as it would be typed after the **model** command (for example `phabs*powerlaw`), *surrounded by braces*. The braces are required: they group the expression — which usually contains spaces and operators — into a single command argument; without them **compare** would read its separate words as separate arguments. Parameters whose names match a uniquely named parameter of the current model are carried over from the current best fit; everything else takes its `model.dat` default.

`@`**<file.xcm>** a candidate read from a model file saved earlier with the **save** `model` command. The file builds that candidate's full model *including its own parameter values, limits, freezes and priors*, giving complete per-model control. Sourcing is model-only: **data**, **response**, **statistic** and **ignore** lines in the file are skipped so the noticed data and statistic are left unchanged.

Shared parameters are matched *by name*, and only names that are unique within each model qualify — the ambiguous `norm` carried by every additive component is deliberately skipped. A comparison is meaningful only under a fixed statistic and identical noticed data, so an alternative that changes the number of noticed channels is reported and skipped.

Default (information criterion). With no leading keyword, each alternative is fit with the current **method** and ranked by the Akaike and Bayesian information criteria,

$$\mathrm{AIC} = \mathrm{stat} + 2k, \qquad \mathrm{BIC} = \mathrm{stat} + k \ln N, \tag{6.1}$$

where k is the number of free parameters and N the number of noticed channels. The table reports the statistic, degrees of freedom, and Δstat, ΔAIC, ΔBIC *relative to the best-fit (lowest-statistic) row*. Because the information criteria

penalise free parameters, a simpler model with a comparable statistic can have a *negative* ΔAIC/ΔBIC — i.e. it is preferred despite a marginally worse fit. A shared-parameter shift table follows, showing how each common parameter moves between models.

The `global` keyword fits each alternative with **fit global** (a derivative-free global optimisation by Differential Evolution, followed by a polish with the current method) instead of a plain local fit. This guards against a stuck local minimum in one candidate corrupting its AIC/BIC; the table is otherwise identical to the default.

The `nest` keyword replaces the information-criterion table with a ranking by Bayesian evidence (the marginal likelihood $\log Z$, see Chapter 3). The current model and every alternative are explored by nested sampling (**nest**), and the models are ranked by $\log Z$ rather than by AIC/BIC. Unlike AIC/BIC — which score each model at a single point (its best fit) plus a parameter-count penalty — the evidence integrates the likelihood over the prior, so the Occam penalty is the prior volume itself rather than a count. This is the rigorous model-comparison quantity, at the cost of $N + 1$ full nested-sampling runs.

Bayesian inference must be switched on (**bayes** `on`) before **compare** `nest`, and each model's parameters must have proper, integrable priors. Priors are supplied in two ways:

Carried by name (default) the prior type, its hyper-parameters and the soft limits set on the *current* model are copied by name onto each inline alternative's matching parameters. Set the priors you want on the current model (with **bayes**) before running, and they propagate to the shared physical parameters (N_H, `kT`, abundances, same-named norms).

Per-candidate via `@`<file.xcm> a saved-model candidate carries its own priors and limits, so it is the way to give a comparison model priors that differ from the current one. This is the recommended route whenever the priors matter: define each candidate in its own **save** `model` file with the priors already set.

As with a standalone **nest** run, if any parameter would be sampled under a flat prior spanning more than four decades (typical of an unconstrained normalisation with its default limits) **compare** `nest` refuses to run and names the offending parameters. Give each a prior (e.g. **bayes** <par> `jeffreys` on a same-named current-model parameter, or an `@`<file.xcm> candidate that sets it) or tighten its limits, then retry. **compare** `nest` uses the current **nest** settings (**nest live**, **nest tol**, etc.); **nest live** 200 trades $\log Z$ precision for speed when scanning many models, and **parallel nest** N parallelises each run.

The evidence table reports, for each model, $\log Z$ and its uncertainty, $\Delta \log Z = \log Z - \log Z_{\rm best} \leq 0$ relative to the best (highest-evidence) model, $2\,\Delta \log Z$ on the Kass & Raftery (1995) scale (> 2 positive, > 6 strong, > 10 decisive evidence against the model), and the posterior model probability $P(M|D) = Z_i / \sum_j Z_j$ under equal model priors. The reference row (flagged `*`) is the highest-evidence model.

Examples:

```
XSPEC> compare {powerlaw + gaussian} {bbody}
```

Fit `powerlaw+gaussian` and `bbody` against the current data and compare them to the current model by ΔAIC/ΔBIC:

```
Model comparison (statistic = cstat, N = 1024 noticed channels):
   #  model                                stat    dof       dStat        dAIC
   1  powerlaw + gaussian                434.22   1019        0.00       +0.00
   2  powerlaw (current)                 434.22   1022        0.00       -6.00
   3  bbody                           3.472e+07   1022   3.472e+07  +3.472e+07  +3.47
```

Here the added Gaussian does not improve the fit, so the simpler current `powerlaw` has the lower (better) AIC and BIC despite the identical statistic.

```
XSPEC> compare global {tbabs(diskbb+comptt)} {tbabs*relxilllpCp}
```

The same comparison, but each alternative is fit with a global optimiser so a poor local minimum cannot bias the result.

```
XSPEC> bayes on
XSPEC> bayes 2 jeffreys
XSPEC> fit
XSPEC> nest live 400
XSPEC> compare nest {powerlaw} @bbody_prior.xcm
```

Rank the current model and two candidates by Bayesian evidence. The inline `powerlaw` inherits the current model's Jeffreys prior on the normalisation by name; the `bbody_prior.xcm` candidate supplies its own priors:

```
Model comparison by evidence (nested sampling, statistic = cstat):
   #  model                                ln Z       +/-       dlnZ     2 dlnZ   P(M|
```

```
  1  powerlaw                      -238.27     0.636      +0.00      -0.00    0.750  *
  2  powerlaw (current)            -239.37     0.652      -1.10       2.20    0.250
  3  @bbody_prior.xcm            -1.736e+07     0.634 -1.736e+07  3.472e+07    0.000
```

The blackbody is decisively rejected and the two power-law fits share the posterior probability.

PyXspec. The same harness is available as `Fit.compare(alternatives, method='lm'|'global'|'nest')`, which returns the per-model table as a list of dictionaries (the `nest` method adds `logZ`, `logZerr`, `deltaLogZ` and `postProb` to each row).

6.5.4 error (and rerror)

determine confidence intervals of a fit

Determine the confidence region for a model parameter.

Syntax: **error** `[[stopat <ntrial> <toler>] [maximum <redchi>] [nonew] [<delta fit statistic>] [<model param range>...]]`

where `<model param range> ::= [<modelName>:]<first param> - <last param>`

determines the ranges of parameters to be examined, and `<delta fit statistic>` (distinguished from the model parameter indices by the inclusion of a decimal point), is the change in fit statistic used.

For response parameters (see **gain** command), use **rerror** with identical syntax except:

`<response param range> ::= [<sourceNum>:]<first param> - <last param>`

The **error** command uses one of two algorithms. If Monte Carlo Markov Chains are loaded (see **chain** command) the error range is determined by sorting the chain values, and then taking a central percentage of the values corresponding to the confidence level as indicated by `<delta fit statistic>`. This is likely to be the faster of the two algorithms.

When chains are **not** loaded, **error**'s algorithm is as follows:

Each indicated parameter is varied, within its allowed hard limits, until the value

of the fit statistic, minimized by allowing all the other non-frozen parameters to vary, is equal to the last value of fit statistic determined by the **fit** command plus the indicated <`delta fit statistic`>, to within an absolute (not fractional) tolerance of <`toler`>. Note that before the **error** command is executed, the data must be fitted. The initial default values are the range 1-1 and the <`delta fit statistic`> of 2.706, equivalent to the 90% confidence region for a single interesting parameter. The number of trials and the tolerance for determining when the critical fit statistic is reached can be modified by preceeding them with the `stopat` keyword. Initially, the values are 20 trials with a tolerance of 0.01 in fit statistic.

If a new minimum is found in the course of finding the error, the default behavior is to abort the calculation and then automatically rerun it using the new best fit parameters. If autosaving is enabled (see **autosave**), the new best fit is written to the autosave file before the calculation restarts, so it is recoverable even if the rerun is interrupted. If you prefer not to automatically rerun the **error** calculation, then enter `nonew` at the start of the command string. The rerun only happens if the new fit is genuinely better: if re-fitting with all parameters free does not lower the fit statistic by more than the critical delta of the **method** command, **error** reports that and stops instead of restarting. If you see this warning, try refitting with a smaller critical delta. The maximum keyword ensures that **error** will not be run if the reduced chi-squared of the best fit exceeds <`redchi`>. The default value for <`redchi`> is 2.0.

Since there are very many scenarios which may cause an **error** calculation to fail, it is highly recommended that you check the results by viewing the 9-letter error string, which is part of the output from the **tclout error** command (see **tclout** for a description of the error string). If everything went well, the error string should be "FFFFFFFFF".

Examples:

```
//Assume that the current model has four model parameters.

XSPEC> error 1-4
//Estimate the 90% confidence ranges for each parameter.
XSPEC> error 9.0
//Estimate the confidence range for parameters 1-4 with delta fit
// statistic = 9.0, equivalent to the 3 sigma range.
XSPEC> error 2.706 1 3 1. 2
//Estimate the 90% ranges for parameters 1 and 3, and the 1. sigma
// range for parameter 2.
XSPEC> error 4
//Estimate the 1. sigma range for parameter 4.
XSPEC> error nonew 4
```

```
//Same as before, but calculation will NOT automatically restart
//if a new minimum is found.
XSPEC> error stop 20,,3
//Estimate the 1-sigma range for parameter 3 after resetting the
//number of trials to 20.Note that the tolerance field had to be
//included (or at least skipped over).
```

6.5.5 fit

fit data

Find the best fit model parameters for the current data by minimizing the current statistic.

Syntax: **fit** <fit method parameters>

The arguments to fit depend on the fitting method currently in use. See the **method** command for details (and for the usage of the USE_CHAIN_RULE option in the user's startup Xspec.init file). Output from the **fit** command also depends on the fitting method currently in use.

Using the Levenberg-Marquardt algorithm, the parameters accepted are the maximum <number of iterations> before the user is prompted, the <critical delta>, which is the (absolute, not fractional) change in the statistic between iterations less than which the fit is deemed to have converged, and <critical beta>.

The <critical beta> provides an optional second stopping criterion, and it refers to the |beta|/N value reported during a Levenberg-Marquardt fit. This is the norm of the vector derivatives of the statistic with respect to the parameters divided by the number of parameters. At the best fit this should be zero, and so provides another measure of how well the fit is converging. <critical beta> is set to a negative value by default, which renders it inactive.

Including the string `delay` as an argument to fit turns on delayed gratification. It is turned off by `nodelay`. Delayed gratification modifies the way the damping parameter is set and has been shown in many cases to speed up convergence. The default is `nodelay`.

If <number of iterations>, <critical delta>, <critical beta>, `delay`, or `nodelay` is entered through the **fit** command, it also becomes the future default value for the currently loaded fit **method** (ie. Levenberg-Marquardt).

Examples:

```
XSPEC> fit
// Fit with the default number of iterations and critical delta
// chi-squared.
XSPEC> fit 60
// Fit with 60 as the number of iterations.
XSPEC> fit  50 1.e-3
// Fit with 1.e-3 as the critical delta.
XSPEC> fit 50 1.e-3  20.
// Same fit, but will now use |beta|/N = 20.0 as another stopping
// criterion in addition to that of the critical delta.
XSPEC> fit delay
// Same fit, but will now use delayed gratification.
```

Global optimization (the `global` keyword)

Syntax: **fit** `global [<maxgen> [<popsize>]]`

A plain **fit** is a *local* optimizer: from the current parameter values it descends to the nearest minimum of the statistic. On a multi-modal or degenerate parameter space — common when fitting several physically distinct models to the same data, or a model with near-degenerate components — it can converge to a local minimum that is far from the global best fit, with the result depending on the starting point.

Including the keyword `global` (any three-or-more-character prefix of `global`) prefixes the fit with a derivative-free *global* search. XSPEC runs a Differential Evolution sweep over the soft-limit box of every thawed parameter, moves to the best point it finds, and then polishes that point with the current **method** (Levenberg–Marquardt, `migrad`, or `simplex`). Because the search uses only statistic evaluations and never derivatives, it works for the entire model library, including local models that provide no analytic gradients. The final statistic, parameter errors, and covariance are produced by the ordinary polish, so they are identical in form to those of a normal **fit**.

The search box is the soft ("min"/"max") limit range of each parameter, which is always finite, so **fit** `global` never refuses to start. A positive parameter whose range spans more than four decades (a column density or normalisation, say) is searched in $\log_{10}$ so the wide scale does not swamp the others; a parameter with a soft minimum of zero and a very wide range draws a non-fatal warning, since its scale cannot be inferred. The optional <maxgen> sets the number of Differential Evolution generations (default 200) and <popsize> the population

size (default automatic, clamp$(10D, 15, 200)$ for D thawed parameters); these are the only tunable controls.

fit `global` is *never-regress*: if the global search plus polish does not improve on the statistic of the current fit, the original parameters are restored and a message is printed, so running it on an already-good fit cannot make it worse. The run is reproducible under **xset seed**, and the Differential Evolution likelihood batch can be spread over N processes with **parallel global** <N>.

After the fit, any *other* distinct minima within Δstat < 9 (about 3σ for one degree of freedom) of the best fit are reported as competitive basins, with their Δstat and parameter values. These flag genuine degeneracies that a single best-fit number hides; the suggestion is to test them as distinct models with **compare**. The basin list is also available in PyXspec as `Fit.globalBasins` after `Fit.globalFit()`.

Example:

```
XSPEC> fit global
 ... Differential Evolution search, then a normal polish ...

 fit global: 1 additional competitive basin(s) within delta-stat < 9
 of the best fit (stat = 243.95):
    basin       dStat          stat   parameters(in fit order)
       2      3.1846        247.14   2.0014, 0.00039876, ...
  (test these as distinct models with ‘compare‘.)
XSPEC> fit global 300 40
// Run a longer global search: 300 generations, population 40.
```

6.5.6 freeze (and rfreeze)

set parameters as fixed

Do not allow indicated model parameters to vary. (See also **thaw**)

Syntax: **freeze** [<param range>...]

where <param range> ::= [<modelName>:]<param#>|<param#> - <param#>.

For response parameters (see **gain** command):

Syntax: **rfreeze** [<param range>...]

6.5.8 goodness

perform a goodness of fit Monte-Carlo simulation

Perform a Monte Carlo calculation of the goodness-of-fit.

Syntax: **goodness** [<# of realizations>] [sim|nosim] [fit|nofit]

This command simulates <# of realizations> spectra based on the model and writes out the percentage of these simulations with the test statistic less than that for the data. If the observed spectrum was produced by the model then this number should be around 50%. This command only works if the sole source of variance in the data is counting statistics. The sim|nosim switch determines whether each simulation will use parameter values drawn from a Gaussian distribution centered on the best fit with sigma from the covariance matrix. The sim switch turns on this option, nosim turns it off in which case all simulations are drawn from the best-fit model. The default starting setting is sim. The fit|nofit switch determines whether fitting will be performed on each simulated dataset to find the best fit before calculation of the test statistic. The fit switch turns on this option, nofit turns it off in which case the test statistic is calculated immediately after creating the simulated dataset. The default starting setting is fit. The goodness calculation may be sped up by setting the **parallel** command's goodness option to run multiple processes.

When the data form a covariance group fit with **statistic** chistokes or chicov, each simulated dataset is drawn jointly from the per-bin cross-spectrum covariance, so the Monte Carlo distribution of the test statistic is correct; see the correlated-spectra discussion in the Statistics appendix.

6.5.9 hmc

run a Hamiltonian Monte Carlo (NUTS) chain

Syntax: **hmc** <subcommand> [<args>]

The **hmc** command runs a No-U-Turn-Sampler variant of Hamiltonian Monte Carlo over the currently-thawed model parameters, using the analytic posterior gradient described in Appendix H.1. Unlike **chain**, which steps the chain via Metropolis–Hastings or Goodman–Weare proposals, **hmc** integrates Hamiltonian trajectories with leap-frog steps adapted to the local posterior curvature. This is typically more sample-efficient than **chain** on smooth, moderate-

dimensional posteriors but each step is more expensive because every leap-frog sub-step requires one gradient evaluation.

hmc does not require every component to be gradiented. A fully gradiented model uses the fast reverse-mode (VJP) gradient; if one or more additive or multiplicative components lack a registered gradient their parameters are finite-differenced on the statistic while the rest stay analytic (the same per-column hybrid used by **fit**). It refuses to start only on a hard structural reject — an ungradiented convolution component, or a mixing (`mix`, `amx`) model — in which case the user is asked to fall back to **chain**.

A **fit** should be performed before **hmc run** so the starting point is at or near the maximum-likelihood estimate; the warmup phase then adapts the leap-frog step size and the diagonal mass matrix. See the discussion of HMC in Chapter 3 and the worked example in Section 3.4 for context and example output.

By default **hmc** runs all chains back-to-back in the parent process. To run them in parallel as separate sub-processes (necessary for an honest Rubin–Gelman $\hat{R}$ diagnostic, see **chain** `stat`), set **parallel hmc** <N> to the number of chains before invoking **hmc run**.

`run` <fileName> Executes the sampler and writes samples to <fileName> as a FITS table compatible with **chain** `load`. Prefix the filename with `!` to discard any existing per-chain checkpoint and overwrite the output file. Without the prefix, if checkpoint files `<fileName>.chain<C>.ckpt` exist from an earlier interrupted run and are dimensionally compatible with the current model, **hmc run** resumes those chains from the saved sample index (skipping warmup), and the checkpoints are deleted on successful completion.

`samples` <N> Number of post-warmup samples per chain. Default 1000.

`warmup` <N> Number of warmup samples per chain. Default 1000. During warmup the leap-frog step size is adapted by dual-averaging and the diagonal mass matrix is fit from sample variance; both freeze at the end of warmup and are held fixed for the production samples.

`chains` <N> Number of independent chains to run. Default 4. Chains share the model and starting point but use distinct RNG streams. Concurrency is gated by **parallel hmc**; with the default **parallel hmc** `1` the chains run back-to-back in the parent process, which is suitable for production sampling but not for $\hat{R}$ (because chains are seeded sequentially rather than independently).

`stepsize` <eps> Initial leap-frog step size. Default 0.05. Adapted by dual-averaging during warmup; if `adapt` is `off` the user value is held fixed throughout.

`maxdepth` <K> Maximum depth of the NUTS tree-doubling recursion. Default 10. A trajectory of depth K corresponds to up to 2^K leap-frog substeps, so the default caps the per-sample cost at 1024 gradient evaluations. Setting this lower trades sampling quality for speed; warnings are emitted at chatter ≥ 10 when trajectories hit the cap.

`target_accept` <X> Target Metropolis acceptance probability for the dual-averaging adaptation. Default 0.8 (Stan default). Raise toward 0.95 for posteriors with strong curvature variation where the default rate produces divergent transitions; expect a smaller step size and a higher per-sample cost.

`divergence_threshold` <X> Hamiltonian-energy error above which a trajectory is flagged as divergent and discarded. Default 1000. Divergent transitions indicate the leap-frog integrator is breaking the symplectic guarantee of HMC, usually because the step size is too large for the local curvature. The total divergent count is printed at the end of a run; if it exceeds a few percent of the post-warmup samples the posterior should be reparameterised or `target_accept` raised.

`adapt on|off` Enables (default) or disables warmup adaptation of step size and mass matrix. With `off` the sampler runs at the user-set `stepsize` with an identity mass matrix; useful for controlled experiments and reproducing earlier runs.

`checkpoint` <N> Write a per-chain checkpoint every <N> samples. Default 200. Set `0` to disable checkpoint writing (and therefore resume).

`info` Print the current sampler settings, the divergent count from the most recent run if any, and mean Metropolis acceptance.

Examples:

```
XSPEC> fit
XSPEC> parallel hmc 4
XSPEC> hmc warmup 1000
XSPEC> hmc samples 1000
XSPEC> hmc chains 4
XSPEC> hmc run mychain.fits
```

The samples are written to `mychain.fits` in a format compatible with **chain** `load`, so subsequent **error**, **flux** and **eqwidth** commands can use them as the parameter-distribution source.

6.5.10 improve

search for a better fit near the current one

Syntax: improve

The **improve** command tries to escape a local minimum without restarting the fit from scratch. It is the warm-restart counterpart of **fit** `global`: starting from the current best fit, it runs a short derivative-free Differential Evolution pass seeded in a shell *around* the current parameter values, then polishes the best point it finds with the current **method**. Use it when a **fit** has converged but you suspect it has settled in a local rather than the global minimum — **improve** perturbs the parameters outward, re-optimizes, and keeps the new result only if it is better.

improve requires a valid **fit** first (it warm-starts from the current parameters). Like **fit** `global` it is *never-regress*: if the search does not beat the current statistic the original parameters are restored, so it can only help. It works with every fit **method** (in earlier versions **improve** was available only for the Minuit methods; it now uses the same Differential Evolution machinery as **fit** `global` for all of them). The run can be interrupted with Ctrl-C, which leaves the pre-command fit in place.

The difference from **fit** `global` is the scope of the search. **improve** explores a *local* shell about the current fit, so it is cheaper and is meant for refining a fit or hopping a nearby barrier; **fit** `global` searches the full soft-limit box of every parameter from scratch and is the tool for a genuinely global, multi-modal problem. As with **fit** `global`, any additional competitive minima within $\Delta\text{stat} < 9$ of the best fit are reported and are available in PyXspec as `Fit.globalBasins`.

Example:

```
XSPEC> fit
// Levenberg-Marquardt converges to a local minimum.
XSPEC> improve
// Differential Evolution shell about that fit, then a polish; if a
// deeper minimum is found the parameters move there, otherwise they
// are left unchanged.
```

6.5.11 margin

MCMC probability distribution

Use the currently loaded MCMC chains to calculate a multi-dimensional probability distribution.

Syntax: **margin** <step spec.> [<step spec.> ...]

where <step spec.> ::= [log|nolog] [<model name>:]<fit param index> <low value> <high value> <no. steps>.

The indicated fit parameter is stepped from <low value> to <high value> in <no. steps>+1 trials. The stepping is either linear or log. Initially, the stepping is linear but this can be changed by the optional string log before the fit parameter index. nolog will force the stepping to be returned to the linear form. The number of steps is set initially to ten. If the parameter number is zero then the fit statistic values are used. The results of the most recently run margin command may be examined with **plot margin** (for 1-D and 2-D distributions only). This command does not require that spectral data files are loaded, or that a valid fit must exist.

Examples:

Assuming chain(s) are loaded consisting of 4 parameters.

```
XSPEC>margin 1 10.0 12.0 20 log 3 1.0 10.0 5
// Calculate a 2-D probability distribution of parameter 1 from
// 10.0-12.0 in 20 linear bins, and parameter 3 from 1.0-10.0 in
// 5 logarithmic bins.
XSPEC>margin 2 10.0 100.0 10 nolog 4 20. 30. 10
// Now calculate for parameter 2 in 10 log bins and
// parameter 4 in 10 linear bins.
```

6.5.12 nest

run a nested-sampling exploration

Syntax: **nest** <subcommand> [<args>]

The **nest** command runs a multi-ellipsoidal nested sampler over the currently-thawed model parameters. Unlike **chain** and **hmc**, nested sampling produces a sequence of *weighted* posterior samples plus an estimate of the marginal likelihood (model evidence) $\log Z$, computed by likelihood-restricted prior sampling inside a contracting envelope of bounding ellipsoids. See the nested-sampling

discussion in Chapter 3 for the algorithmic background and Section 3.4 for a worked example.

nest requires proper, integrable priors on every thawed parameter. Parameters with default flat priors over a hard range that spans more than four decades (typical of column densities and normalisations) cause **nest** to refuse to start; in such cases either narrow the parameter limits via **newpar** or assign a Jeffreys prior via **bayes** `JEFFREYS` before invoking **nest run**. See Chapter 3, Section 3, for the available prior families.

nest does not warm-start from a fit – it samples from the prior and contracts inward. A **fit** is therefore not required before **nest run**, but it is useful for diagnosing the prior choice: if the MLE lies in a region the prior assigns very low probability the run will spend many iterations in the contraction phase before finding the likelihood peak.

By default **nest** runs in a single process. Set **parallel nest** <N> to run with N worker processes sharing the likelihood-restricted sampling pool, which gives sub-linear but significant speed-up.

`run` <fileName> Executes the sampler and writes the output to <fileName>. The file contains two FITS extensions: a `NEST` table with the weighted nested samples and their log-likelihood / log-weight columns, and a `CHAIN` table with equal-weight resampled draws compatible with **chain** `load`. Prefix the filename with `!` to discard any existing checkpoint and overwrite. Without the prefix, if a checkpoint file `<fileName>.ckpt.fits` exists and is dimensionally compatible with the current model, **nest run** resumes from the saved iteration count.

`live` <N> Number of live points (the size of the contracting ellipsoid pool). Default 400. More live points give a finer-grained estimate of $\log Z$ and reduce the chance of missing isolated modes, at proportional cost. Conventional values run from 100 (cheap exploration) to 2000 (production-grade evidence for paper figures).

`tol` <eps> Skilling log-evidence termination tolerance. Default 0.5. Sampling stops when the residual evidence in the live points falls below <eps> times the accumulated total. Smaller values run longer and produce tighter $\log Z$ estimates.

`enlarge` <factor> Enlargement factor applied to the volume of the bounding ellipsoids before drawing new live points. Default 1.2. Larger values reduce the chance of clipping the true iso-likelihood contour at the cost of more sampling per replaced live point.

`clusters` <K> Maximum number of ellipsoids used to bound the live-point cloud. Default 16. Setting `1` forces the single-ellipsoid algorithm of Mukher-

jee et al. (2006), which is faster but cannot follow multi-modal posteriors; the default uses the X-means split heuristic to grow the ellipsoid count up to the cap as needed.

`resample` <N> Number of equal-weight posterior draws to write to the `CHAIN` extension. Default 4000. These are produced by importance resampling from the weighted `NEST` table and are what **chain** `load` will see if it is pointed at the output file.

`checkpoint` <N> Write a checkpoint every <N> iterations. Default 200. Set `0` to disable checkpoint writing.

`seed` <N> Seed of the sampler's own random stream. Default `0`, which makes each **nest run** draw a fresh seed; the seed used is printed at the end of the run and reported by **nest info**, so a run made this way can still be repeated afterwards by setting `seed` to that value. The seed covers the resampled `CHAIN` extension as well as the sampler itself, so repeating a run reproduces the whole output file. Nested sampling does not draw from the generator **xset** `seed` controls, so this setting is sufficient on its own – two runs with the same seed agree even if that generator was disturbed in between. Note that **parallel nest** <N> partitions the sampling across workers, so reproducing a run also requires the same worker count.

`info` Print the current sampler settings, and the seed the most recent run used.

`evidence` Print the $\log Z$ estimate and its uncertainty from the most recent run.

Examples:

```
XSPEC> newpar 1 0.05 0.001 1e-6 1e-6 100000 1e6
XSPEC> newpar 5 0.04 0.01  1e-30 1e-30 1e20 1e24
XSPEC> bayes on
XSPEC> bayes 1 JEFFREYS
XSPEC> bayes 5 JEFFREYS
XSPEC> parallel nest 4
XSPEC> nest live 400
XSPEC> nest run nest.fits
XSPEC> nest evidence
```

The **newpar** commands narrow the search ranges of the absorber column and apec normalisation (otherwise their default 31-decade ranges would cause **nest** to refuse to start), and the **bayes** `JEFFREYS` commands switch them to log-uniform priors so that the search proceeds over the full narrowed range without favouring large absolute values.

6.5.13 renorm

renormalize model to minimize statistic with current parameters

Renormalize model, or change renorm conditions.

Syntax: **renorm** [auto|none|prefit]

The **renorm** command will adjust the normalizations of the model by a single multiplication factor, which is chosen to minimize the fit statistic. Such a renorm will be performed explicitly whenever the command is used without a key-word, or during certain XSPEC commands, as determined by the following key-words:

auto	Renormalize after a **model** or **newpar** command, and at the beginning of a fit.
prefit	Renormalize only at the beginning of a fit.
none	Perform no automatic renormalizations, i.e., only perform them when a **renorm** command is given explicitly.

This command works by doing an analytic calculation to change the normalizations of all additive components in the model to minimize the fit statistic. The relative normalizations of the components will not be changed. This analytic calculation cannot be performed for all the fit statistics, in particular it will not work for cstat if there is a background file. If the analytic calculation is not possible this command will have no effect.

6.5.14 sim

bulk posterior-predictive simulation

Generate many simulated realizations of the loaded spectra, optionally refit each one and run a command file on it, and accumulate the results. This is the in-program replacement for the common scripted loop of **tclout** `simpars`, **newpar** and **fakeit** (see the **multifake** script): it is faster, keeps the whole random-number sequence under a single **xset** `seed`, and can be run in parallel.

Syntax: **sim** <# of realizations> [fsigma <f>] [nostat] [fit] [@<cmdfile>] [outfile <name>]

For each realization, **sim** draws a set of parameter values, evaluates the model,

```
   // All 5 spectra now use ks as the test statistic.
XSPEC>statistic chi churazov 5
   // Spectrum 5 will use chi-sq using the churazov weighting
```

6.8.5 xsect

set the photoionization cross-sections

Change the photoelectric absorption cross-sections in use.

Syntax: **xsect** [bcmc|obcm|vern]

The three options are: `bcmc`, from Balucinska-Church & McCammon (1992) with a new He cross-section based on Yan et al. (1998); `obcm`, as `bcmc` but with the old He cross-section, and, `vern`, from Verner & Yakovlev (1995) and Verner et al. (1996). This changes the cross-sections in use for all absorption models with the exception of **wabs**, the **tbabs** family of models, amd **ismabs**.

6.8.6 xset

set variables for XSPEC models

Modify a number of XSPEC internal switches.

Syntax: **xset** [abund |cosmo |delta |mdatadir |method
|seed |statistic |usechainrule |weight
|xsect |<string_name>] [<options>
|<string_value>]

The arguments `abund, cosmo, method, statistic, weight`, and `xsect` just run the appropriate XSPEC commands. `mdatadir` changes the directory in which XSPEC searches for model data files. You probably don't want to change this. The `seed` option requires an integer argument, which will then be used to immediately re-seed and re-initialize XSPEC's random-number generator.

The `delta` option is for setting fit delta values (see the **newpar** command) which are proportional to the current parameter value rather than fixed. For example,

```
XSPEC> xset delta .15
```

will set each parameter fit delta to .15 * parVal. To turn proportional deltas off and restore the original fixed deltas, set `delta` to a negative value or 0.0. The current proportional delta setting can be seen with **show control**.

The `usechainrule` option can be used to switch between the fast (usechainrule yes) and slow (usechainrule no) options when calculating the derivatives of the fit statistic.

The <`string_name`> option can be used to pass string values to models. XSPEC maintains a database of <`string_name`>, <`string_value`> pairs created using this command. Individual model functions can then access this database. Note that **xset** does no checking on whether the <`string_name`> is used by any model so spelling errors will not be trapped.

To access the <`string_name`>, <`string_value`> database from within a model function use the fortran function `fgmstr`. This is defined as `character*128` and takes a single argument, the string name as a `character*128`. If the <`string_name`> has not been set then a blank string will be returned.

In addition to the model control strings listed further below, a number of operational switches use the same database mechanism. These are toggled with

```
XSPEC> xset CONTROL_STRING yes
```

where the value may be any of `yes/no/on/off/true/false/1/0` (case-insensitive). All of them default to off, and a change takes effect immediately – there is no need to reload data or models. The switches whose names end in `_CROSS` or `_SELFTEST` are validation aids: they run two independent implementations of the same calculation and warn if the results disagree, at some cost in speed. The `DISABLE_` switches revert an optimized numerical path to its slower reference implementation.

DISABLE_ANALYTIC_GRAD	Revert leven and migrad fits to finite-difference derivatives of the fit statistic even when every active model component has a registered analytic gradient. Useful to reproduce fit results from versions before analytic gradients were introduced, or to test whether a change in a fit result is due to derivative accuracy. HMC always uses its own analytic path and ignores this switch.
HYBRID_GRAD_CROSS	In leven fits, calculate the fit derivatives with both the analytic path and finite differences at every iteration and print a comparison, with per-parameter detail at higher chatter levels. The fit itself proceeds with the analytic result.
DISABLE_VJP	Replace the fused vector-Jacobian product used for HMC gradients with the explicit Jacobian-then-multiply path. Intended for comparing the two analytic implementations.
VJP_SELFTEST	At the start of an HMC run, cross-check the vector-Jacobian product against the product of the full Jacobian with a random adjoint vector for each active model, and refuse to run HMC if they disagree. Intended for use when developing new model gradient functions.
VJP_CROSS	At every HMC gradient call compute both the vector-Jacobian product and a finite-difference estimate and warn on disagreement. More exhaustive, and considerably more expensive, than VJP_SELFTEST.

SVD_CROSS	For responses using the low-rank (SVD) convolution path, also run the exact matrix multiplication on every call and warn if the two disagree beyond the expected reconstruction accuracy.
DISABLE_CONVOLVEMANY	Revert the batched convolution of analytic-gradient Jacobian columns to the legacy one-column-at-a-time loop. Intended for benchmarking only.
CONVOLVEMANY_CROSS	Cross-check the batched convolution of analytic-gradient Jacobian columns against the one-column-at-a-time loop on every gradient call and warn on the first disagreement.
DISABLE_COMPONENT_CACHE	Recompute every model component on each evaluation instead of reusing cached component results.
COMPONENT_CACHE_CROSS	Recompute model components in a scratch buffer and verify that the cached result matches.
ARF_PREBAKE_CROSS	Cross-check the prebaked response-times-ARF fast path against a live multiplication by the ARF.

The current <`string_name`> options, models to which they apply and brief descriptions are given in the following table :

APECROOT or SPEXROOT	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)expcheb6, (b)(v)(v)gadem, (v)mekal, mkcflow, nlapec, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Switch from default AtomDB or SPEX input files.
APECTHERMAL or SPEXTHERMAL	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Thermally broaden emission lines in APEC or SPEX input files.

APECVELOCITY or SPEXVELOCITY	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Velocity broaden emission lines in APEC or SPEX input files.
APECMINFLUX or SPEXMINFLUX	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Set minimum flux for line broadening when using APEC or SPEX input files.

APECBROADPSEUDO or SPEXBROADPSEUDO	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Broaden pseudo-continuum lines when using APEC or SPEX input files.
APECNOLINES or SPEXNOLINES	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Continuum only when using APEC or SPEX input files.

APEC_TRACE_ABUND or SPEX_TRACE_ABUND	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Set trace element abundances when using APEC or SPEX input files.
APECLOGTINTERP or SPEXLOGTINTERP	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Use logarithmic interpolation between tabulated temperatures when using APEC or SPEX input files.

APECMULTITHREAD or SPEXMULTITHREAD	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Parallelize over temperatures when calculating the lines in the spectrum using the APEC or SPEX input files.
APECEEBREMSS or SPEXEEBREMSS	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Include calculation of the e-e bremsstrahlung when using the APEC or SPEX input files.

APECDOECOR or SPEXDOECOR	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)equil, (b)(v)expcheb6, (b)(v)(v)gadem, (b)(v)(v)gnei, (v)mekal, mkcflow, (b)(v)(v)nei, nlapec, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Include the electron-density correction term (off by default) when using the APEC or SPEX input files.
APECUSESPEX	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)(v)cie, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)expcheb6, (b)(v)(v)gadem, (v)mekal, mkcflow, nlapec, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Switch from APEC to SPEX input files but apply all other APEC settings.

APECUSENEI	(b)(v)(v)apec, c6(v)mekl, c6p(v)mekl, ce(v)mkl, (b)(v)cempow, (b)(v)cheb6, (b)(v)coolflow, (b)(v)(v)dem, (b)(v)expcheb6, (b)(v)(v)gadem, (v)mekal, mkcflow, nlapec, (b)snapec, (b)(v)(v)tapec, rs(v)(v)apec, vmcflow	Use NEI code to calculate CEI spectra.
APECEIGENFILE	(b)(v)equil, (b)(v)(v)gnei, (b)(v)(v)nei, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov	Switch from default AtomDB NEI eigenvector file.
NEIAPECROOT	(b)(v)equil, (b)(v)(v)gnei, (b)(v)(v)nei, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov	Switch from default AtomDB NEI input files.
NEIVERS	(b)(v)equil, (b)(v)(v)gnei, (b)(v)(v)nei, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov	Select the AtomDB NEI version. Ignored (with a warning) if NEIAPECROOT is also set.

NEI_TRACE_ABUND	(b)(v)equil, (b)(v)(v)gnei, (b)(v)(v)nei, (b)(v)(v)npshock, (b)(v)(v)pshock, (b)(v)(v)rnei, (b)(v)(v)sedov	Set the trace element abundances for the NEI models (the NEI counterpart of APEC_TRACE_ABUND). Value is a number or an element symbol whose abundance the trace elements follow.
NSA_FILE	nsa	Change filename used for model data.
NSAGRAV_DIR	nsagrav	Change directory used for model data files.
NSMAX_DIR	nsmax	Change directory used for model data files.
NSMAXG_DIR	nsmaxg	Change directory used for model data files.
NSX_DIR	nsx	Change directory used for model data files.
POW_EMIN, POW_EMAX	powerlaw, bknpower, bkn2pow, cutoffpl	Switch to normalize to a flux calculated over an energy range.
CARBATM	carbatm	Switch the directory for the input file.
CFLOW_VERSION	mkcflow, vmclow	Switch CFLOW version number.
CFLOW_NTEMPS	mkcflow, vmclow	Switch number of temperature bins used in CFLOW model.
HATM	hatm	Switch the directory for the input file.
ISMABSROOT	ismabs	Directory for the ismabs input data files.
ISMDUSTROOT	ismdust, olivineabs	Directory for the ismdust and olivineabs input data files.

TBABSVERSION	tbabs, tbfeo, tbgas, tbgrain, tbpcf, tbrel, (v)tbvarabs	Select the version of the TBabs cross-sections and ISM abundances (1 for the original, 2 for the default).
LINECRITLEVEL	gaussian, Lorentzian and Voigt emission- and absorption-line models, gsmooth, lsmooth, rsgauss, feklor, fekblor	Set the critical flux level down to which the line profiles are calculated, overriding each model's built-in default.
IREFLECT_MAX_E	ireflect	Set the maximum energy for which to calculate the output spectrum.
IREFLECT_PRECISION	ireflect	Set the fractional precision for the Greens' function adaptive integration.
REFLECT_MAX_E	reflect	Set the maximum energy for which to calculate the output spectrum.
REFLECT_PRECISION	reflect	Set the fractional precision for the Greens' function adaptive integration.
RFXCONV_DIR	rfxconv	directory to use for input model data files instead of the standard modelData directory.
RFXCONV_MAX_E	rfxconv	the maximum energy for which to calculate the output spectrum.
RFXCONV_PRECISION	rfxconv	precision used in adaptive Gauss-Kronrod quadrature of the Greens' function integral.

RGS_XSOURCE_FILE	rgsext, rgsxsrc	set the file from which to read the image filename, boresight and extraction information.
SUZPSF-IMAGE	suzpsf	Set image file to be used for surface brightness.
SUZPSF-RA	suzpsf	Set RA for center surface brightness map which is taken from the WMAP.
SUZPSF-DEC	suzpsf	Set Dec for center surface brightness map which is taken from the WMAP.
SUZPSF-MIXFACT-IFILE#	suzpsf	Set filename to read mixing factors.
SUZPSF-MIXFACT-OFILE#	suzpsf	Set filename to write mixing factors.
XILCONV_MAX_E	xilconv	maximum energy for which to calculate the output spectrum.
XILCONV_PRECISION	xilconv	precision used in adaptive Gauss-Kronrod quadrature of Greens' function integral.
XILCONV_DIR	xilconv	directory to use for ionization model filenames instead of the standard modelData directory.
XILCONV_VERSION	xilconv	version number of the xillver file, options are 3 and 5 (the default).
XMMPSF-IMAGE	xmmpsf	Set image file to be used for surface brightness.
XMMPSF-RA	xmmpsf	Set RA for center surface brightness map which is taken from the WMAP.